

Digital Signals: ADC and Sensor Interfaces

INS03 · 01386309 Agricultural Instrumentation and IoT · version 9

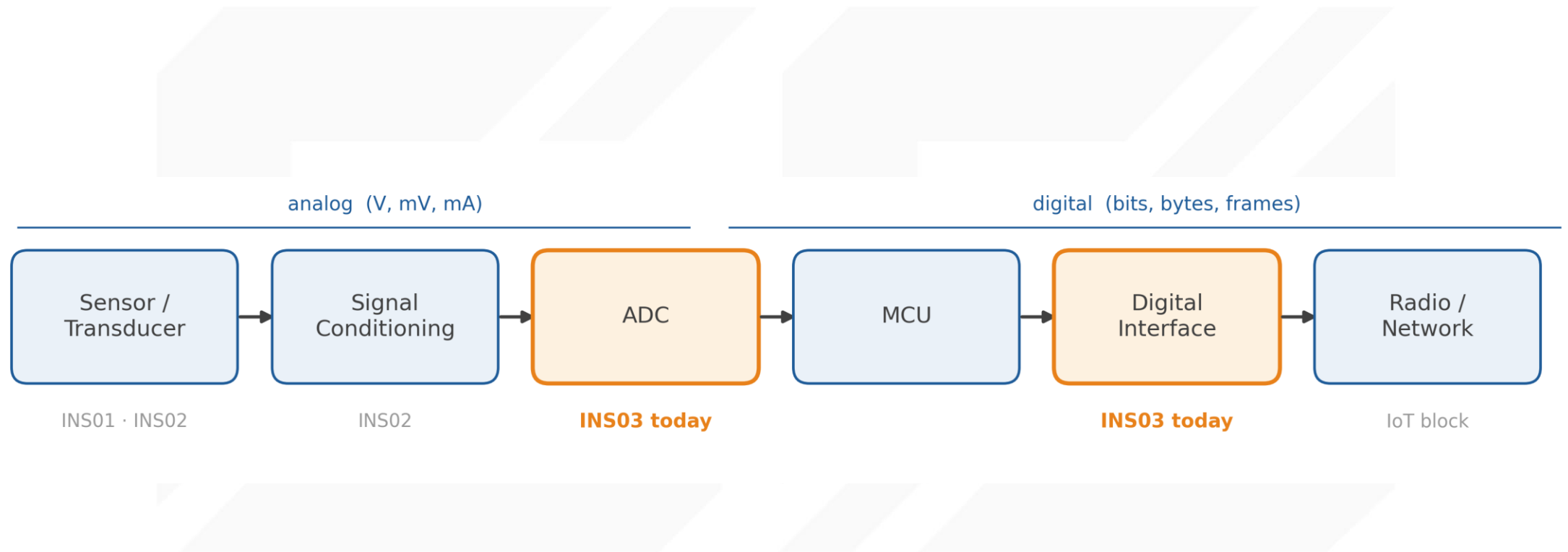
Quantization · DAC · SAR ADC · Sampling · ESP32 ADC and attenuation · UART / RS-485 · I²C · SPI · 1-Wire · CAN

ผู้ช่วยศาสตราจารย์ ดร. วสุ อุดมเพทายกุล

Department of Biosystems and Agricultural Engineering, KMITL

Where Digital Signals Enter the Chain

ก่อนหน้านี้ได้แรงดันที่พร้อมวัดแต่ยังเป็น analog บทนี้ดูว่าแรงดันกลายเป็นตัวเลขได้อย่างไร และตัวเลขส่งต่อกันระหว่างอุปกรณ์อย่างไร



Bits, Bytes and Hexadecimal

ใน lab จะเจอเลขอย่าง 0x57, 0x04, 0x0001 ตลอดเวลา วิธีอ่านคือแปลงทีละหลัก หลักละ 4 บิต ไม่ต้องตั้งหาร

ฐานสิบ	ฐานสอง	hex
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

bit มีค่า 0 หรือ 1 ในวงจรคือ LOW/HIGH หรือ 0 V กับ 3.3 V

byte 8 บิต แทนค่าได้ $2^8 = 256$ ค่า (0–255) และเป็นหน่วยที่ UART, I²C, SPI รับส่งกันทีละก้อน

word 16 บิต แทนค่าได้ 65,536 ค่า ค่าที่อ่านจาก Modbus หรือค่า ADC 12 บิตก็เก็บใน word เดียว

hex 1 หลักแทน 4 บิตพอดี 1 byte จึงเขียนด้วยตัวอักษรแค่ 2 ตัว ในโค้ดใช้ 0x นำหน้า (0x57) ส่วนเอกสารบางเล่มเขียน 57h

ตัวอย่างค่า digital

0x57 = $0101\ 0111_2 = 5 \times 16 + 7 = \mathbf{87}$ I²C address ของ ultrasonic

0x3C = $0011\ 1100_2 = 3 \times 16 + 12 = \mathbf{60}$ I²C address ของ OLED

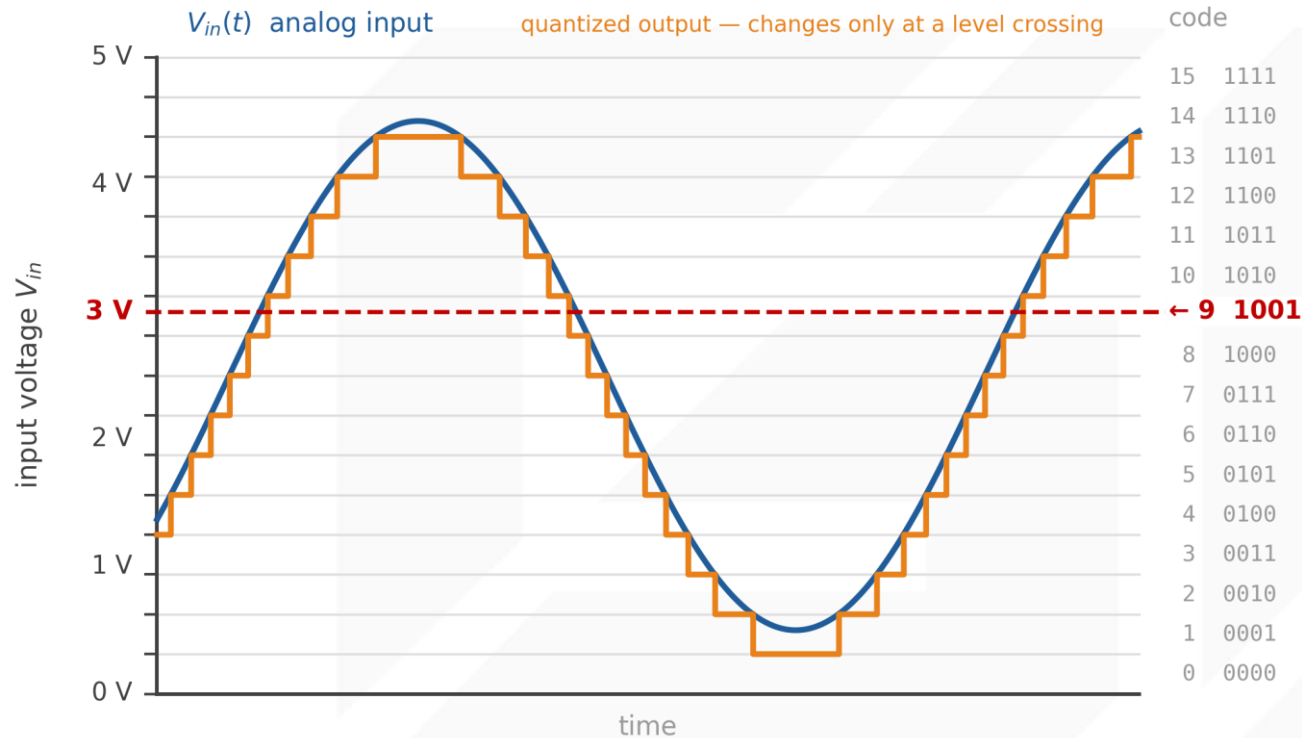
0x010A = $1 \times 256 + 0 \times 16 + 10 = \mathbf{266} \div 10 = 26.6\ ^\circ\text{C}$ (word อุณหภูมิจาก XY-MD02)

0000 1111₂ = **0x0F** = 15 แบ่งทีละ 4 บิตแล้วเทียบกับตารางด้านซ้าย

Quantization: the Amplitude Axis

ADC ขนาด n บิต แบ่งช่วงแรงดันเป็น 2^n ระดับ ระดับละ 1 LSB ค่าที่อยู่ระหว่างระดับจะถูกปัดลง ความกว้างของระดับนี้คือ resolution

$$V_{ref} = 5 \text{ V} \cdot n = 4 \text{ bit} \cdot 2^4 = 16 \text{ levels (code 0-15)} \cdot 1 \text{ LSB} = 5/16 = 0.3125 \text{ V}$$



$$V_{in} = 3.0 \text{ V}: d = \lfloor 3.0 / 5 \times 16 \rfloor = \lfloor 9.6 \rfloor = 9 \rightarrow \text{code } 1001$$

3.0 V lies between level 9 (2.81 V) and level 10 (3.13 V) → the ADC reports the lower level, 9

$$N = 2^n \text{ steps}$$

$$\text{LSB} = \frac{V_{ref}}{2^n}$$

$$d = \left\lfloor \frac{V_{in}}{V_{ref}} 2^n \right\rfloor$$

$$V = \frac{d}{2^n} V_{ref}$$

บิต	ระดับ	พบใน
8	256	DAC ของ ESP32, sensor ราคาถูก
10	1,024	Arduino UNO · ESP8266
12	4,096	ESP32 · Raspberry Pi Pico
16	65,536	ADS1115
24	16.7 ล้าน	HX711 (load cell) · MAX31865

Worked Example — 12-bit ADC, $V_{ref} = 3.3 \text{ V}$

ลองคำนวณด้วยตัวเลขชุดเดียวกับที่จะใช้ในกิจกรรม 4.1 และสังเกตว่า resolution กับ accuracy เป็นคนละเรื่องกัน

$$n = 12, \quad V_{ref} = 3.3 \text{ V}$$

$$\text{LSB} = \frac{3.3}{4096} = 0.81 \text{ mV}$$

$$V_{in} = 2.00 \text{ V}: \quad d = \left\lfloor \frac{2.00}{3.3} \times 4096 \right\rfloor = 2482$$

$$V = \frac{2482}{4096} \times 3.3 = 1.9998 \text{ V}$$

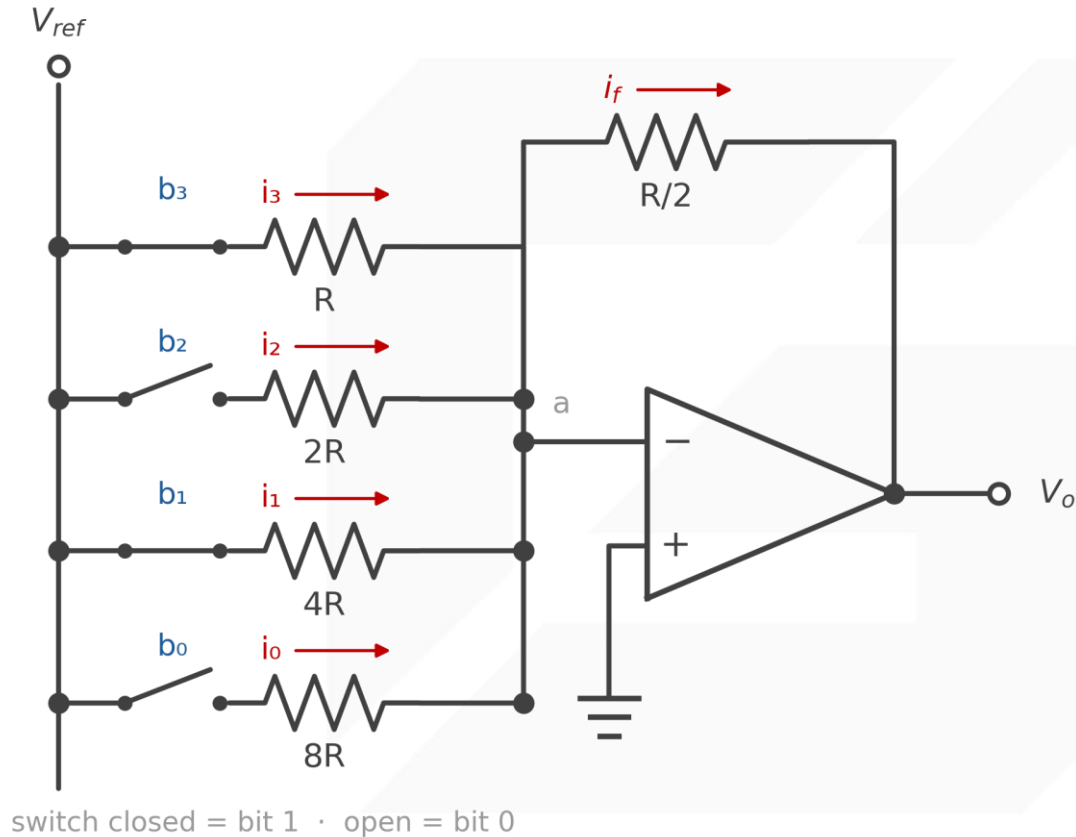
$$V_{in} = 0.30 \text{ V}: \quad d = \lfloor 372.4 \rfloor = 372 \Rightarrow V = 0.2997 \text{ V}$$

Resolution ≠ Accuracy

- resolution 0.8 mV บอกได้ว่า ADC แยกความต่าง 0.8 mV ได้ ไม่ได้บอกว่าค่าที่อ่านถูกต้อง ESP32 ที่ยังไม่ calibrate อ่านคลาดได้ถึง 100–200 mV
- ใน term project ความคลาดเคลื่อน 100 mV ของ capacitive soil sensor หมายถึงความชื้นดินที่ผิดไปหลาย % นี่คือเหตุผลที่กิจกรรม 4.1 ให้ calibrate 6 จุดก่อนใช้

DAC — Bits to Voltage

DAC ทำงานกลับทางกับ ADC ที่ต้องพูดถึงก่อนเพราะข้างใน SAR ADC มี DAC อยู่ด้วย และ ESP32 เองก็มี DAC 8 บิตที่ GPIO25 กับ GPIO26

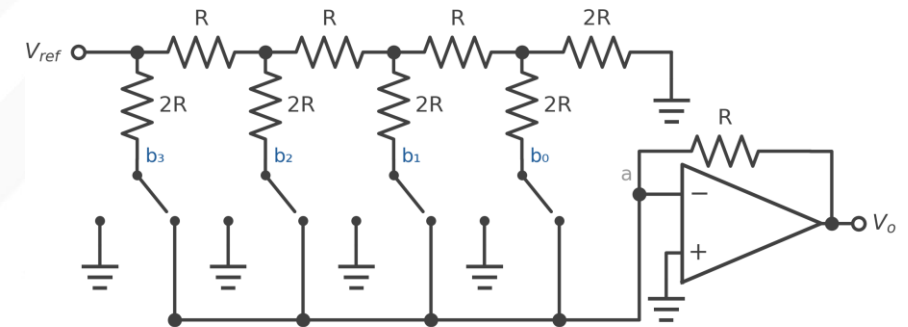


หลักการ บิตไหนเป็น 1 switch ของกิ่งนั้นปิด กระแสจากกิ่งนั้นไหลเข้า node a แล้ว op-amp รวมกระแสทุกกิ่งออกมาเป็นแรงดัน ซึ่งก็คือ summing amplifier ที่เรียนไปใน INS02

น้ำหนักของบิต ค่า $R, 2R, 4R, 8R$ ทำให้ $i_3 = 2i_2 = 4i_1 = 8i_0$ บิตที่สูงกว่าจึงมีน้ำหนักเป็นสองเท่าของบิตถัดไปพอดี

ข้อจำกัด ถ้าจะทำ 16 บิต ตัวต้านทานตัวเล็กสุดกับตัวใหญ่สุดต้องต่างกัน $2^{15} = 32,768$ เท่า และทุกตัวต้องแม่นยำเท่ากัน ซึ่งผลิตจริงไม่ได้

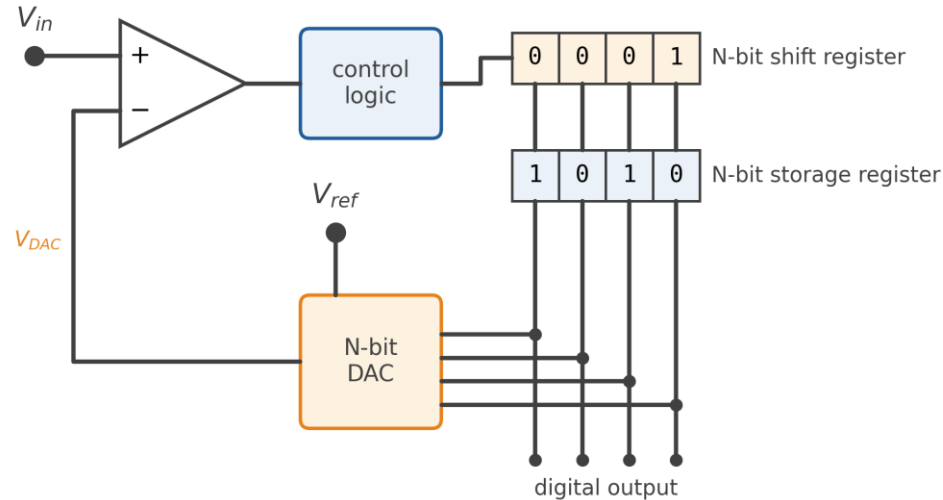
ทางออก ใช่วงจร R-2R ladder (รูปล่าง) ซึ่งใช้ตัวต้านทานแค่สองค่าไม่ว่าจะกี่บิต ที่มาของสมการดูได้ในภาคผนวก A1-A4



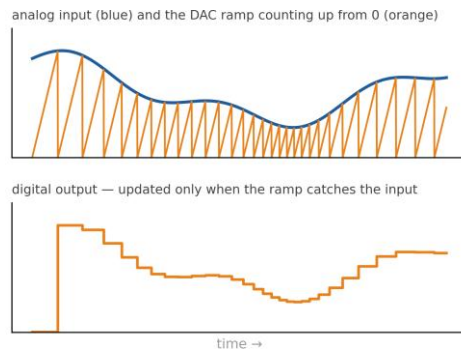
right contact (1) = node a · left contact (0) = ground · only two resistor values, R and $2R$

Basic ADC — Comparator, DAC and a Register

ADC แบบ feedback ทุกชนิดมีโครงสร้างเดียวกัน คือ comparator เปรียบ V_{in} กับ V_{DAC} แล้ว control logic ปรับ register จนกว่าผลเปรียบเทียบจะเปลี่ยน



comparator: is $V_{in} > V_{DAC}$? → control logic updates the register until the answer flips



Comparator รับ V_{in} ที่ขา + และ V_{DAC} ที่ขา - ให้ output เป็น 1 บิต คือเป็น 1 เมื่อ $V_{in} > V_{DAC}$

N-bit DAC แปลงค่าใน register กลับเป็นแรงดัน V_{DAC} โดยเทียบกับ V_{ref} (ดูแผ่นที่ 6)

Control logic ดูผลจาก comparator แล้วตัดสินใจว่าจะปรับค่าใน register อย่างไร วิธีปรับตรงนี้แหละที่ทำให้ ADC แต่ละแบบต่างกัน

Register 2 ชุด shift register เก็บค่าที่กำลังลองอยู่ ส่วน storage register เก็บผลลัพธ์สุดท้ายไว้ให้ MCU อ่าน ระหว่างที่รอบถัดไปกำลังทำงาน

สองกลยุทธ์ของ control logic

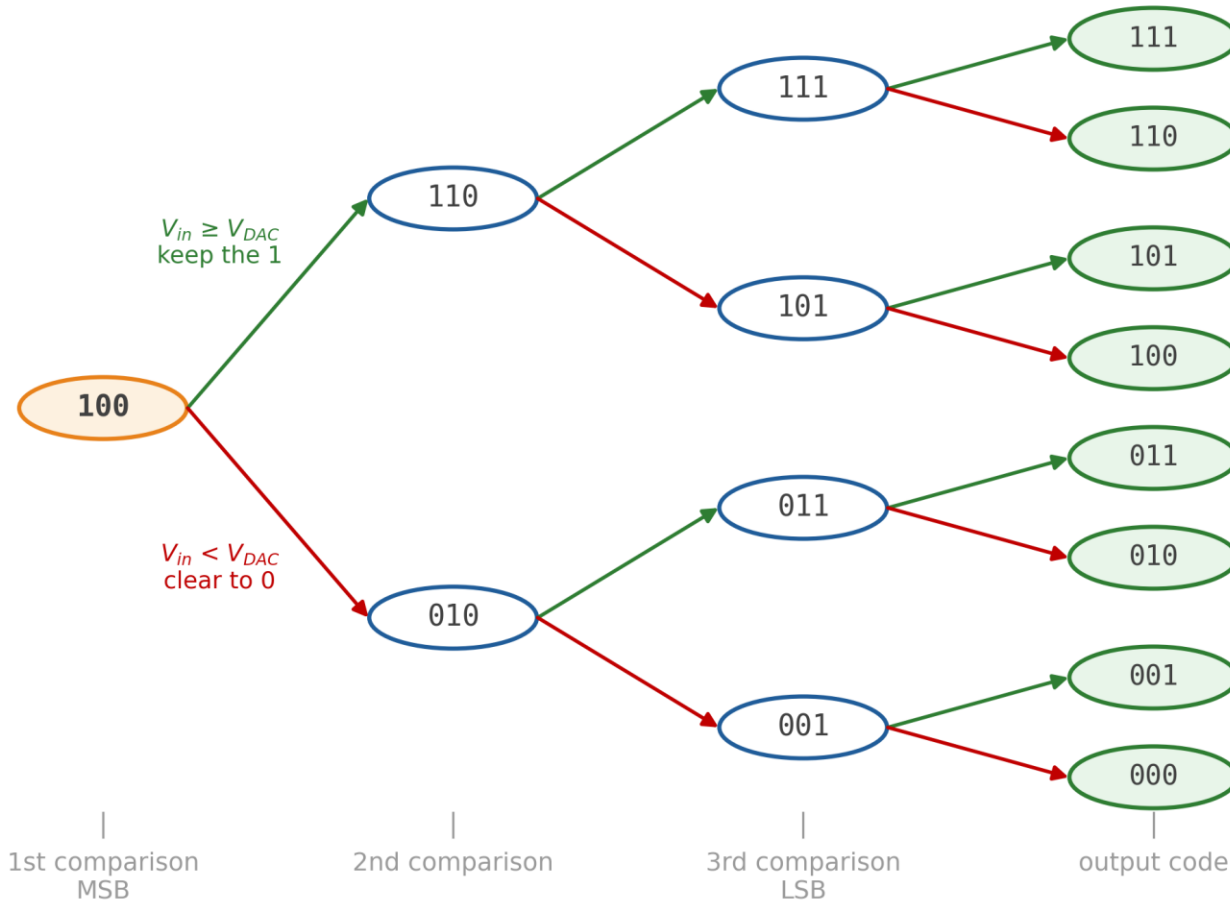
Digital-ramp ADC ใช้ counter นับขึ้นทีละ 1 จากศูนย์ จนกว่า V_{DAC} จะไล่ทัน V_{in} เข้าใจง่ายแต่ช้า เพราะค่าใหญ่ต้องนับถึง 2^N ครั้ง และเวลาแปลงแต่ละครั้งไม่เท่ากัน (กราฟซ้าย)

SAR ลองทีละบิตจาก MSB ใช้แค่ N รอบเสมอ เป็นวิธีที่ดีกว่าและจะพูดถึงในสองแผ่นถัดไป

SAR as a Binary Search

แทนที่จะนับขั้นทีละ 1 SAR ใช้วิธีหาแบบ "มากไปหรือน้อยไป" ทุกครั้งที่เทียบ ช่วงของคำตอบจะลดลงครึ่งหนึ่ง

3-bit SAR: each comparison halves the remaining range — 3 steps reach one of 8 codes



ดัดแปลงจากแผนภาพต้นไม้ของ Sheel (2013)

เริ่มที่กลางช่วง code แรกที่ลองคือ 100 ซึ่งเท่ากับ $V_{ref}/2$ ผลเทียบครั้งแรกจะบอกว่าคำตอบอยู่ครึ่งบนหรือครึ่งล่าง

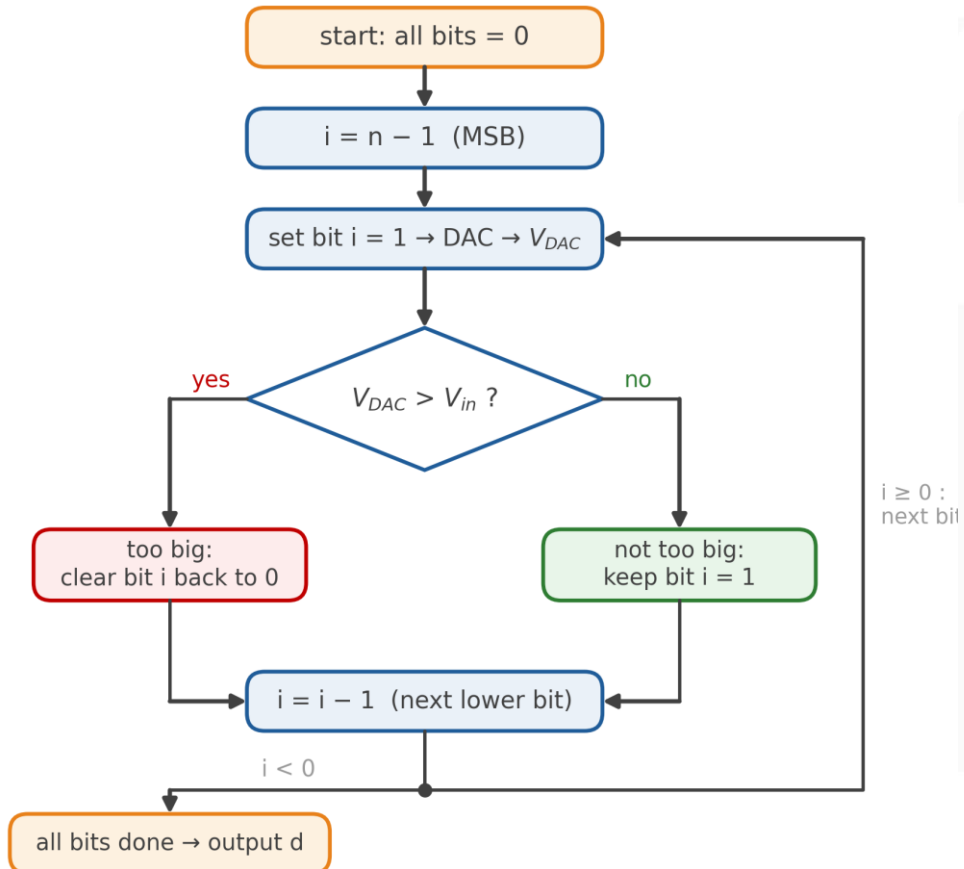
ทางเดินในต้นไม้ ถ้ายังไม่เกินให้เดินขึ้น (เก็บบิตเป็น 1) ถ้าเกินให้เดินลง (ล้างบิตเป็น 0) เส้นทางจากรากถึงใบก็คือ code ที่ได้

จำนวนขั้น ต้นไม้ลึก n ชั้นครอบคลุม code ได้ 2^n ค่าพอดี จึงเทียบแค่ n ครั้ง ต่างจาก Flash ADC ที่ต้องใช้ comparator $2^n - 1$ ตัวพร้อมกัน

ดูต่อในแผ่นถัดไป ตัวอย่าง 4 บิตในแผ่นที่ 9 ก็คือการเดิน ขึ้น → ลง → ขึ้น → ลง บนต้นไม้ 4 ชั้น จนได้ใบ 1010

Successive-Approximation ADC (SAR)

ADC แบบที่อยู่ใน ESP32, Arduino และ MCU แทบทุกตัว หลักการคือลองทีละบิตจาก MSB ลงมา คล้ายการชั่งของด้วยตุ้มน้ำหนักทีละลดลงทีละครึ่ง



ตัวอย่าง 4 บิต $V_{ref} = 15\text{ V}$ $V_{in} = 10.1\text{ V}$ (ขั้นละ $15/16 = 0.9375\text{ V}$)

ขั้น	ลองตั้งบิต	V_{DAC}	เทียบ V_{in}	ผล
i = 3	1000	$8/16 \times 15 = 7.50\text{ V}$	$7.50 < 10.1$ ไม่เกิน	เก็บ $b_3 = 1$
i = 2	1100	$12/16 \times 15 = 11.25\text{ V}$	$11.25 > 10.1$ เกิน	ล้าง $b_2 = 0$
i = 1	1010	$10/16 \times 15 = 9.375\text{ V}$	$9.375 < 10.1$ ไม่เกิน	เก็บ $b_1 = 1$
i = 0	1011	$11/16 \times 15 = 10.31\text{ V}$	$10.31 > 10.1$ เกิน	ล้าง $b_0 = 0$

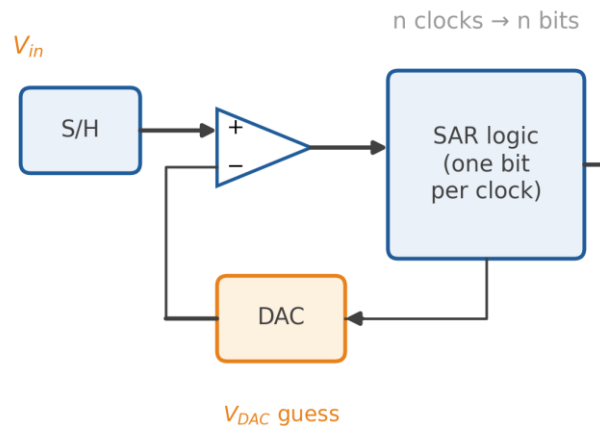
ผลลัพธ์ $1010_2 = 10 \rightarrow V = 10/16 \times 15 = 9.375\text{ V}$ คลาดไม่เกิน 1 LSB (0.94 V)

- ADC n บิตใช้ n ขั้นตอนเสมอ 12 บิตก็ 12 ขั้น ไม่ว่า V_{in} จะเป็นเท่าไร จึงเร็วพอสำหรับ MCU และกินพื้นที่ชิปน้อย
- ต้องมีวงจร sample-and-hold ค้าง V_{in} ไว้ระหว่างที่หาย ถ้า V_{in} เปลี่ยนกลางคัน ผลจะผิดโดยไม่มีอะไรเตือน
- ESP32 มี SAR ADC 12 บิต 2 ชุด (ADC1 กับ ADC2) เทียบกับ V_{ref} ภายในชิปประมาณ 1.1 V ซึ่งเป็นที่มาของเรื่อง attenuation ในแผ่นที่ 15

ADC Families (1) — SAR · Flash · Pipeline

ADC มีหลายสถาปัตยกรรม ขึ้นกับว่าต้องการกี่บิตและกี่ sample ต่อวินาที สามแบบในแผ่นนี้เน้นความเร็ว

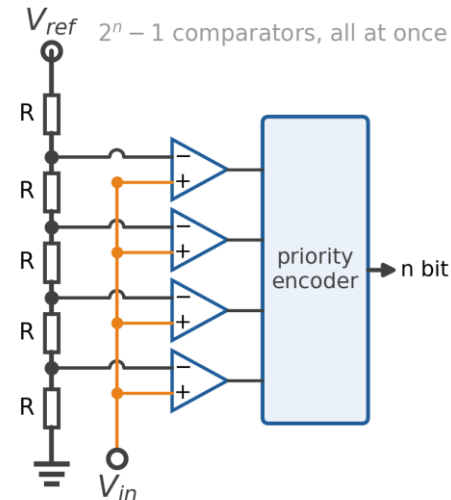
SAR



ทายทีละบิตด้วย DAC ภายใน n บิตใช้ n clock ได้ 8–16 บิต
ความเร็วปานกลาง (kHz–MHz) เป็นจุดสมดุลที่ดีที่สุดสำหรับ MCU
แต่ต้องมี sample-and-hold

พบใน: ESP32 · Arduino · Raspberry Pi Pico

Flash

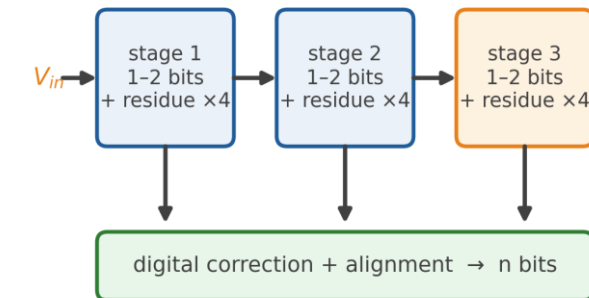


ใช้ comparator $2^n - 1$ ตัวเทียบพร้อมกันในรอบเดียว จึงเร็วที่สุด
(ระดับ GHz) แต่จำนวน comparator เพิ่มขึ้นสองเท่าทุกบิต ทำ
ได้แค่ 6–8 บิต แพงและกินไฟ

พบใน: oscilloscope · video

Pipeline

like an assembly line: a new sample enters every clock,
the result comes out a few clocks later (latency)



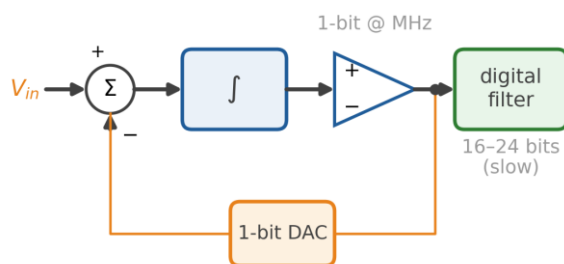
แบ่งงานเป็นหลาย stage ชิ้นละไม่กี่บิต แล้วส่งส่วนที่เหลือไปชั้น
ถัดไปเหมือนสายพาน รับ sample ใหม่ได้ทุก clock ได้ 10–16 บิต
ที่ความเร็วสูง (MHz) แต่ผลออกช้าไปหลาย clock (latency)

พบใน: ระบบสื่อสาร · กล้อง

ADC Families (2) — Delta-Sigma · Dual-slope

สองแบบนี้ยอมช้าเพื่อแลกกับ resolution สูงและทน noise ได้ดี จึงพบในเครื่องมือวัดและ sensor ความละเอียดสูง

Delta-Sigma ($\Delta\Sigma$)

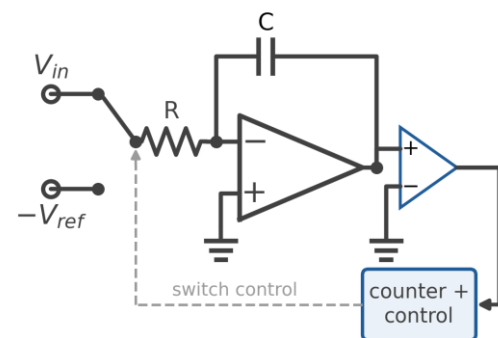


the loop forces the average of the 1-bit stream to equal V_{in} ; noise is pushed to high frequency and filtered out

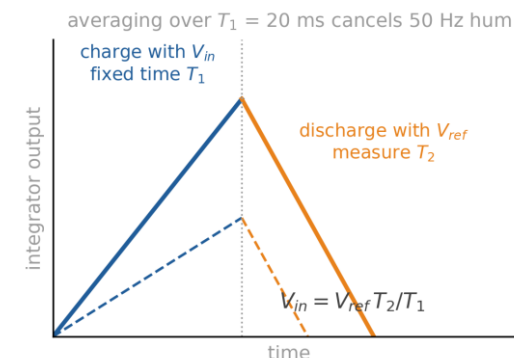
sample ถี่มากด้วย comparator แค่ 1 บิต แล้วให้ digital filter เฉลี่ยจนได้ 16–24 บิต ช้า (Hz–kHz) แต่ตัด noise ได้ดีมาก เหมาะกับสัญญาณระดับ mV ที่เปลี่ยนแปลงช้า

พบใน: HX711 (load cell) · ADS1115 · MAX31865

Dual-slope



ramp up with V_{in} for fixed T_1 , then switch to $-V_{ref}$ and ramp down; the counter measures T_2 until the comparator sees zero



จากวงจรด้านซ้าย switch ต่อ V_{in} เข้า integrator (R กับ C รอบ op-amp) เป็นเวลาคงที่ T_1 แล้วสลับไปต่อ $-V_{ref}$ ให้ output ไหลกลับลงมาถึงศูนย์ comparator จะสั่งหยุด counter (กราฟขวา) เวลา T_2 ที่ได้แปรตาม V_{in} โดยไม่ขึ้นกับค่า R และ C ช้ามากแต่ถูก และถ้าตั้ง $T_1 = 20$ ms noise 50 Hz จะเฉลี่ยหายไปเอง

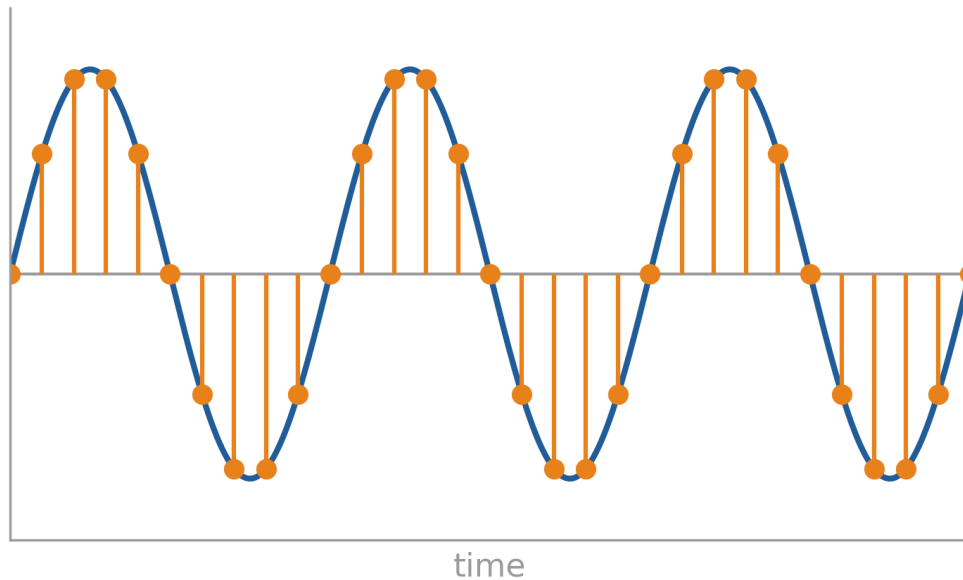
พบใน: multimeter (DMM) · เครื่องชั่ง

- งานเกษตรส่วนใหญ่สัญญาณเปลี่ยนแปลงช้าแต่ต้องการ resolution สูงและทน noise ดังนั้นถ้าวัดสัญญาณระดับ mV เช่น load cell หรือ thermocouple ใช้ $\Delta\Sigma$ ภายนอกอย่าง HX711 หรือ ADS1115 มักได้ผลดีกว่า SAR ที่อยู่ใน MCU

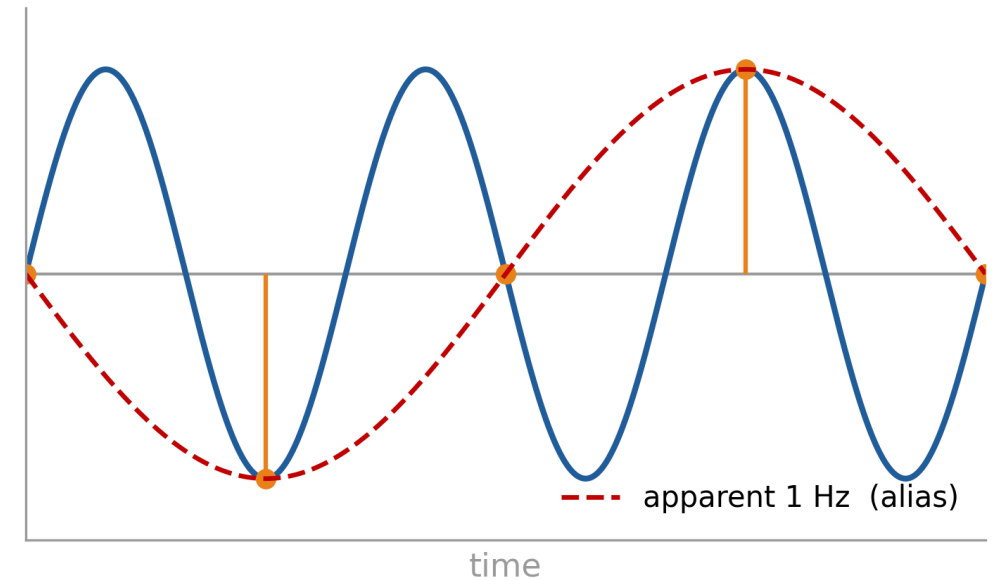
Sampling: the Time Axis

quantization ที่ดูไปในแผ่นที่ 4 คือเรื่องแกนขนาด แผ่นนี้เป็นอีกแกนหนึ่งคือแกนเวลา ที่จะเก็บค่ากี่แค่ไหน ก่อนจะไปได้ ADC จริงใน ESP32

$f_s = 30 \text{ Hz} (> 2f)$ — samples follow the 3 Hz signal



$f_s = 4 \text{ Hz} (< 2f)$ — aliasing: a false 1 Hz signal appears



$$f_s > 2 f_{max}$$

ตามทฤษฎีของ Nyquist sample rate f_s ต้องมากกว่าสองเท่าของความถี่สูงสุดในสัญญาณ ไม่อย่างนั้นจะได้ความถี่ปลอมที่เรียกว่า aliasing ซึ่งกรองออกทีหลังไม่ได้

- sensor ทางการเกษตรส่วนใหญ่ เช่น อุณหภูมิดินหรือความชื้น เปลี่ยนช้ามาก sample ทุก 1-10 นาทีก็พอแล้ว ที่ต้องระวังคือ pulse จาก flow meter และ noise 50 Hz จากไฟบ้าน ซึ่งต้องกรองด้วย low-pass (INS02) ก่อน sample

ESP32 ADC — the Datasheet View

ก่อนต่อ sensor เข้า ESP32 ต้องรู้สเปคเหล่านี้ก่อน จุดที่นักศึกษาสับสนบ่อยที่สุดมีสามเรื่อง คือ ADC2, V_{ref} และ attenuation

รายการ	ค่าตาม datasheet	ความหมายในทางปฏิบัติ
ชนิด · resolution	SAR 12 บิต (0–4095) · ตั้ง 9–12 บิตด้วย <code>analogReadResolution()</code>	1 ชั้นประมาณ 0.8 mV ที่ช่วง 3.3 V แต่อย่าพึ่งเชื่อตัวเลขนี้จนกว่าจะดูแผ่นที่ 16
ช่อง	18 ช่อง: ADC1 = GPIO32–39 (8 ช่อง) · ADC2 = GPIO0, 2, 4, 12–15, 25–27	ADC2 ใช้วงจรร่วมกับ WiFi พอเปิด WiFi แล้วจะอ่านค่าไม่ได้ ใน term project จึงต้องใช้ ADC1 เท่านั้น (กิจกรรม 4.1 ใช้ GPIO34)
V_{ref}	ประมาณ 1.1 V สร้างขึ้นในชิป แต่ละชิปไม่เท่ากัน อยู่ระหว่าง 1.0–1.2 V	ค่าที่อ่านได้ผูกกับ V_{ref} ของชิปตัวนั้น ถ้าไม่ calibrate ก็คลาดได้ถึง $\pm 10\%$ ทันที
ช่วง V_{in}	0 ถึง V_{ref} ที่ ADC core ขยายช่วงได้ด้วย attenuation 0 / 2.5 / 6 / 11 dB	ตั้งด้วย <code>analogSetAttenuation()</code> ดูแผ่นที่ 14–15
ความเป็นเชิงเส้น	ดีในช่วงกลาง แต่เพี้ยนตอนใกล้ 0 V และใกล้เพดาน	ต่ำกว่าประมาณ 0.1 V อ่านได้ศูนย์ และเกินประมาณ 2.45 V (ที่ 11 dB) กราฟจะโค้งงอ
sample rate	สูงสุดประมาณ 100 kS/s ต่อช่องถ้าใช้ driver ของ ESP-IDF ส่วน <code>analogRead()</code> ช้ากว่านั้นมาก	เกินพอสำหรับ sensor เกษตร เอาเวลาที่เหลือไปอ่านซ้ำหลาย ๆ ครั้งแล้วเฉลี่ยเพื่อลด noise ดีกว่า

- กติกาของวิชานี้คือวัดจริงก่อนคำนวณ ค่า count ที่ได้เป็นแค่ตัวเลขดิบ ต้องเอา V_{ref} , attenuation และผล calibrate ของบอร์ดตัวนั้นมาประกอบ ถึงจะแปลงเป็นแรงดันได้

Decibel: a Quick Recap from INS02

ก่อนจะอ่านคำว่า "11 dB" ให้เข้าใจ ต้องนึกออกก่อนว่า dB เป็นอัตราส่วนบน log scale ไม่ใช่หน่วยของแรงดัน

$$B = \log_{10}\left(\frac{P_2}{P_1}\right)$$

$$\text{dB} = 10 \log_{10}\left(\frac{P_2}{P_1}\right)$$

$$P = \frac{V^2}{R} \quad (\text{same load } R)$$

$$\text{dB} = 10 \log_{10}\left(\frac{P_2}{P_1}\right) = 20 \log_{10}\left(\frac{V_2}{V_1}\right)$$

$$\frac{P_2}{P_1} = 10^{\text{dB}/10}$$

$$\frac{V_2}{V_1} = 10^{\text{dB}/20}$$

dB	อัตราส่วนแรงดัน	วิธีจำ
0	$\times 1.00$	เท่าเดิม
2.5	$\times 1.33$	ADC_2_5db ของ ESP32
3	$\times 1.41$	$\sqrt{2}$
6	$\times 2.00$	สองเท่า — ADC_6db
10	$\times 3.16$	$\sqrt{10}$
11	$\times 3.55$	ADC_11db (ชื่อเดิม)
12	$\times 3.98 \approx 4$	ADC_12db (ชื่อใหม่ใน ESP-IDF 5)
20	$\times 10.0$	สิบเท่า
-3	$\times 0.707$	corner ของ RC filter (INS02)
-6	$\times 0.50$	ครึ่งหนึ่ง

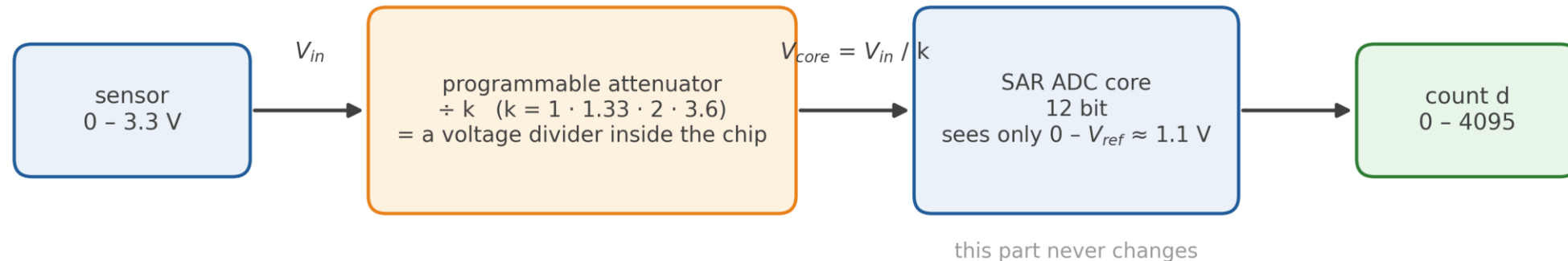
แล้วเกี่ยวกับ ADC อย่างไร

ค่าบวกคือ gain (INS02) ค่าลบคือ attenuation ดังนั้น "attenuation 11 dB" ของ ESP32 หมายถึงแรงดันถูกลดทอนลง $10^{11/20} \approx 3.55$ เท่าก่อนเข้า ADC core ค่าจริงในชิปใกล้ 12 dB (ประมาณ 4 เท่า) Espressif จึงเปลี่ยนชื่อเป็น DB_12 ใน ESP-IDF 5

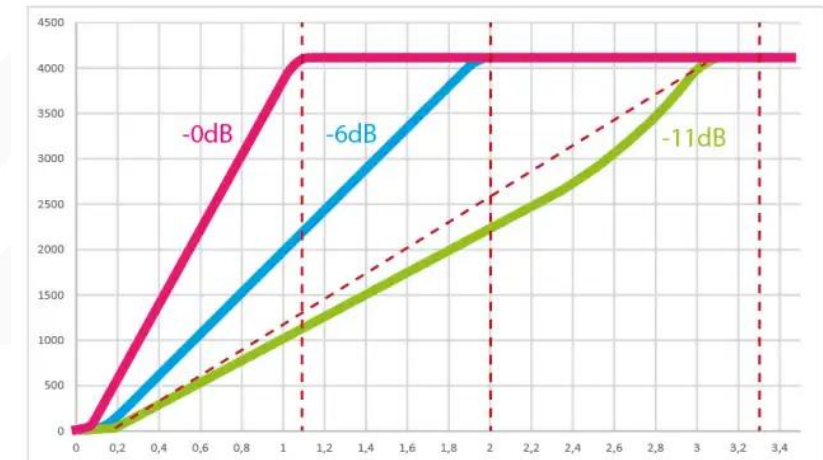
Attenuation = a Voltage Divider Inside the Chip

ADC core วัดได้แค่ 0–1.1 V และเปลี่ยนไม่ได้ ที่เลือกได้คือ voltage divider ที่วางอยู่ข้างหน้า ซึ่งก็คือวงจรเดียวกับที่ตัวเองใน Lab 1

ADC_0db · ADC_2_5db · ADC_6db · ADC_11db (software setting)



ค่าใน Arduino	dB	k	1.1 V × k	ช่วงเชิงเส้น
ADC_0db	0	× 1.0	1.1 V	100 – 950 mV
ADC_2_5db	2.5	× 1.33	1.5 V	100 – 1250 mV
ADC_6db	6	× 2.0	2.2 V	150 – 1750 mV
ADC_11db (DB_12)	11 (≈12)	× 3.6 (≈4)	3.9 V (ขาด 3.3 V)	150 – 2450 mV



$k = 10^{\text{dB}/20}$ ดังนั้น 11 dB ไม่ได้แปลว่า "3.3 V" แต่แปลว่าหารด้วย 3.6 และเชิงเส้นดีแค่ถึง 2.45 V เท่านั้น

ผลวัดจริงจากบอร์ด ESP32 จะเห็นว่าเส้น 11 dB (สีเขียว) โค้งงอตั้งแต่ประมาณ 2.5 V ขึ้นไป และทุกเส้นอ่านได้ศูนย์เมื่อต่ำกว่าประมาณ 0.1 V

ที่มา: L. Llamas, "ESP32 ADC", luisllamas.es/en/esp32-adc/

Same Voltage, Different Count

count ดิบไม่มีความหมายถ้าไม่รู้ attenuation และเมื่อสัญญาณเกินช่วง ADC จะอ้อมตัวเจียบ ๆ นี่คือเหตุผลที่กิจกรรม 4.1 ต้อง calibrate ด้วย DMM

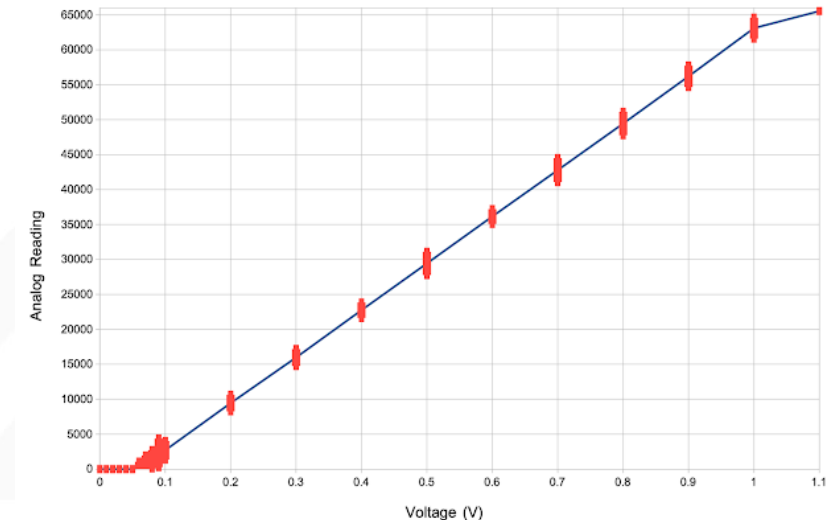
$$V_{in} = 2.0 \text{ V}, 6 \text{ dB}: V_{core} = 1.0 \text{ V} \Rightarrow d \approx \frac{1.0}{1.1} \times 4095 = 3723$$

$$V_{in} = 2.0 \text{ V}, 11 \text{ dB}: V_{core} = 0.56 \text{ V} \Rightarrow d \approx 2097$$

$$V_{in} = 3.15 \text{ V}, 6 \text{ dB}: V_{core} = 1.58 \text{ V} > 1.1 \text{ V} \Rightarrow d = 4095$$

สิ่งที่ได้จากสามบรรทัดข้างบน

- count 3723 กับ 2097 คือแรงดันเดียวกัน ค่าดิบจึงต้องอ่านคู่กับ attenuation และ V_{ref} ของชิปตัวนั้นเสมอ
- พอเกินช่วง ADC ไม่เดือน count จะค้างที่ 4095 แปลงกลับได้แค่เพดานประมาณ 1.8–2.2 V ไม่ใช่ 3.15 V (เป็นกรณีจริงจาก forum ของ Espressif) ต่างจาก DMM ที่จะขึ้น OL ให้เห็น
- หลักการเลือกย่านคือให้ค่าสูงสุดของ sensor อยู่ราว 60–90 % ของช่วงเชิงเส้น sensor 0–3.3 V ทำได้สองทางคือใช้ 11 dB แล้ว calibrate 6 จุด (กิจกรรม 4.1) หรือใส่ divider ภายนอกลดเหลือ 0–2.2 V แล้วใช้ 6 dB (Lab 1)
- analogReadMilliVolts() ใช้ค่า calibrate จากโรงงานที่เก็บใน eFuse ของชิป ดีกว่า analogRead() แต่ก็ยังควรตรวจสอบกับ DMM อย่างน้อย 2 จุด



ผลวัดจริงจาก ESP32 ที่ 0 dB (0–1.1 V ปรับสเกลเป็น 16 บิต) ต่ำกว่าประมาณ 0.1 V อ่านได้ศูนย์ และเริ่มโค้งเมื่อใกล้ 1.0 V

ที่มา: S. Monk, "Comparing ADCs", doctormonk.com/2024/01/comparingadcs.html, 2024

Why Sensors Went Digital

พอ sensor มี ADC ของตัวเองแล้วส่งตัวเลขมาให้ MCU ก็ไม่ต้องวัดแรงดันเองอีก ทำไมอุตสาหกรรมถึงเลือกทางนี้ มีอยู่สี่เหตุผล

1 MCU ไม่ว่าง

การจับเวลา pulse (HC-SR04, 1-Wire) ต้องให้ MCU ว่างในระดับ μs แต่ทุก node ใน term project ต้องเปิด WiFi หรือ LoRa ที่แทรกงานอยู่ตลอด ผลคือค่าที่อ่านได้กระโดดเป็นครั้งคราว (จะให้เห็นใน Lab 4)

2 Noise

สาย analog ที่ยาวจะรับ noise 50 Hz และ noise จากมอเตอร์เข้ามาด้วย ส่วนตัวเลขที่ส่งเป็น frame มี checksum หรือ CRC กำกับ ถ้าผิดก็ตรวจเจอ ไม่ใช่ค่าเพี้ยนไปเจียน ๆ

3 ระยะทาง

ADC ต้องการสายสั้น แต่ RS-485 ส่งได้ถึง 1,200 m และ 1-Wire ได้หลายสิบลเมตร ซึ่งเป็นระยะทำงานในแปลงต้องการ

4 Sensor บอกสถานะได้

เครื่องข่งส่ง ST/US มาบอกค่านิ่งหรือยัง GPS ส่ง fix quality Modbus ส่ง error code sensor ที่ออกแบบดีจะบอกด้วยว่าค่าที่ส่งมาเชื่อถือได้แค่ไหน

- ข้อแลกเปลี่ยนคือเราต้องรู้จัก "ภาษา" ของแต่ละ interface ซึ่งเป็นเนื้อหาครึ่งหลังของวันนี้

Two Layers of Every Interface

interface ทุกแบบต้องมองแยกเป็นสองชั้น ถ้าไม่แยกจะสับสน เช่นถามว่า UART ส่งได้ไกลแค่ไหน จะตอบไม่ได้เลยถ้าไม่รู้ว่าเป็น TTL หรือ RS-485

Physical layer (ชั้นไฟฟ้า)

- ใช้แรงดันเท่าไรแทน 0 และ 1 (3.3 V, ± 12 V หรือแบบ differential)
- ใช้สายกี่เส้น เส้นไหนใช้ร่วมกันได้
- ส่งได้ไกลแค่ไหน ทน noise ได้แค่ไหน
- ต้องใส่ pull-up หรือ terminator หรือไม่

Protocol layer (ชั้นข้อมูล)

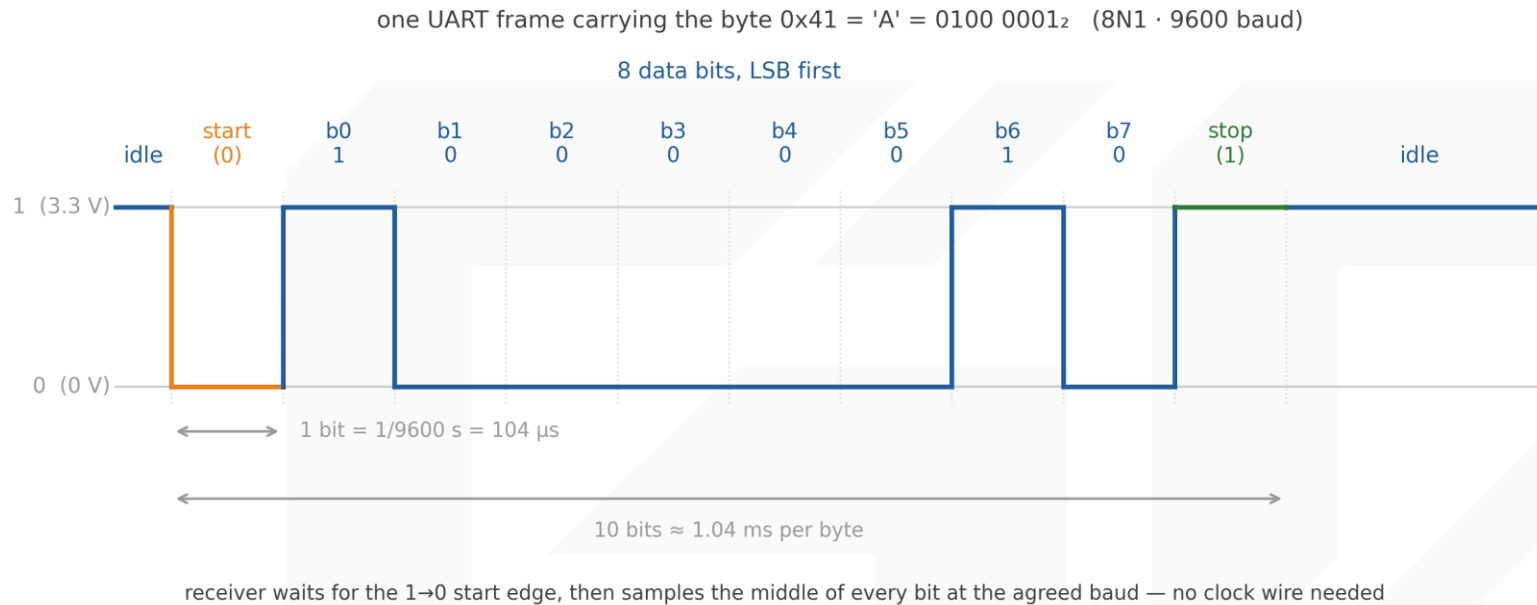
- frame หน้าตาเป็นอย่างไร เริ่มตรงไหน จบตรงไหน ก้อนละกี่บิต
- มี address หรือไม่ และใครเป็นคนกำหนด
- ใครให้ clock มีสาย clock แยก หรือตกลงความเร็วกันไว้ก่อน
- ตรวจสอบความผิดพลาดด้วยอะไร (parity, checksum, CRC, ACK)

สองประโยคที่จำต้องท่อง

- ① ระยะทางเป็นเรื่องของ physical layer ไม่ใช่ protocol frame UART แบบเดียวกันส่งได้ 1 m ถ้าเป็น TTL แต่ส่งได้ 1,200 m ถ้าเป็น RS-485
- ② สิ่งที่ยาก bus (อุปกรณ์หลายตัวบนสายชุดเดียว) ออกจาก point-to-point (ต่อกันหนึ่งต่อหนึ่ง) คือการมี address

UART — the Asynchronous Frame

UART เป็น interface พื้นฐานที่สุด ไม่มีสาย clock จึงต้องตกลง baud rate กันไว้ก่อน และไม่มี address จึงต่อได้แบบ point-to-point เท่านั้น



สาย 2 เส้น TX กับ RX (บวก GND) ต่อไขว้กัน TX ของฝั่งหนึ่งเข้า RX ของอีกฝั่ง

Address ไม่มี จึงเป็น point-to-point

Clock ไม่มีสาย clock ตกลง baud rate กันไว้ก่อน เช่น 9600 หรือ 115200

ระยะ ขึ้นกับ physical layer ดูแผ่นที่ 21

$$t_{bit} = \frac{1}{9600} = 104 \mu s, \quad t_{byte} \approx 10 t_{bit} \approx 1.04 ms$$

- 8N1 หมายถึง 8 data bit, no parity, 1 stop bit เป็นค่าที่เจอในทุก datasheet และเป็น default ของ Serial.begin(9600)
- ถ้าตั้ง baud rate ไม่ตรงกัน จะได้ตัวอักษรขยะ ไม่ใช่ error message เพราะ UART ไม่มีกลไกตรวจสอบ ต่างจาก bus ที่มี ACK
- ESP32 มี UART 3 ชุด คือ Serial (ต่อกับ USB), Serial1 และ Serial2 ใน Lab 4 จะต่อ GPS และ ultrasonic ที่ Serial2
- ส่งสองทิศทางพร้อมกันได้ (full-duplex) เพราะ TX กับ RX เป็นคนละเส้น

UART — What the Bytes Mean

byte ชุดเดียวกันตีความได้สองแบบ คือเป็นข้อความ ASCII ที่อ่านออก หรือเป็นตัวเลขดิบแบบ binary ที่ต้องเปิด datasheet ถึงจะรู้ความหมาย

Weighing scale (RS-232) — ASCII, self-describing, 18 bytes

S	T	,	G	S	,	+			1	0	0	.	0		g	\r	\n
---	---	---	---	---	---	---	--	--	---	---	---	---	---	--	---	----	----

readable without a manual · status ST/US tells whether the value is stable · long

GPS NMEA (TTL UART) — ASCII fields separated by commas + checksum

\$	G	N	G	G	A	,	time	,	lat	,N,	lon	,E,	1	,	08	,...	*hh
----	---	---	---	---	---	---	------	---	-----	-----	-----	-----	---	---	----	------	-----

field 6 = fix quality · field 7 = satellites · *hh = checksum — read quality before using the value

Ultrasonic RCWL-9300A (TTL UART) — binary, 4 bytes

FF	H	L	SUM
----	---	---	-----

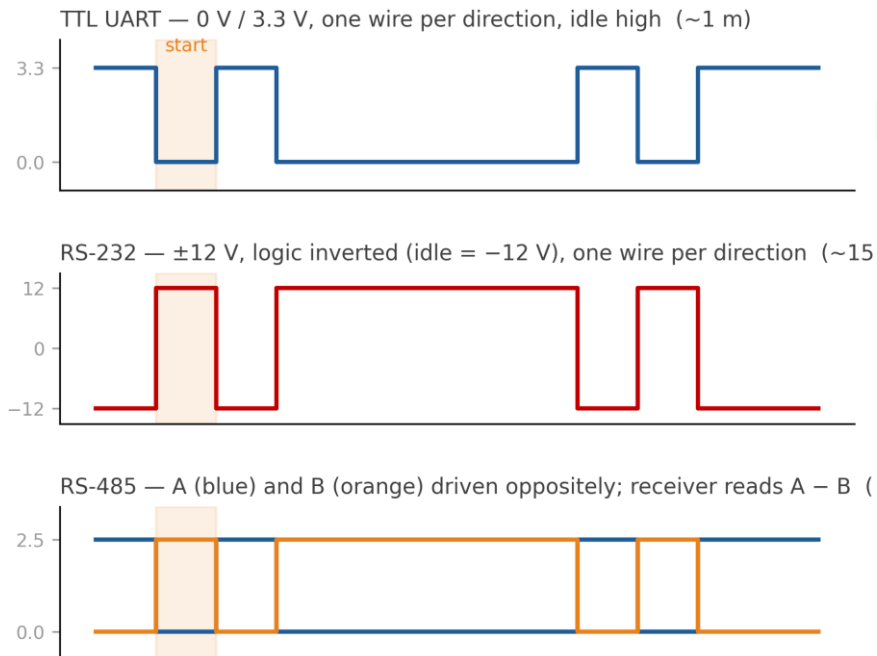
distance = $H \times 256 + L$ (mm) · SUM = (FF + H + L) & FF · compact, but only the datasheet tells you what each byte means

	ASCII	Binary
อ่านด้วยตา	อ่านได้ทันทีใน Serial Monitor	เห็นเป็นตัวอักษรขยะ ต้องเปิดดูในโหมด HEX
ต้องใช้ datasheet	แทบไม่ต้อง เพราะเดารูปแบบได้	ต้องใช้ เพื่อรู้ลำดับ byte, endianness และ scale
ขนาด	ยาว เช่น "+ 100.0 g" ใช้ 9 byte	สั้น 2 byte แทนค่าได้ 0–65,535
ผลใน term project	LoRa มี payload แคประมาณ 50 byte ส่ง ASCII ได้ไม่กี่คำ	ส่งได้หลายค่าต่อ frame กลุ่มที่ใช้ LoRa ควรใช้แบบนี้
เครื่องมือ	USB-TTL กับ Serial Monitor	USB-TTL โหมด HEX หรือ logic analyzer

One Frame, Three Electrical Layers

frame UART เหมือนกันทุกประการ แต่ส่งบนระดับไฟฟ้าคนละแบบ นี่คือตัวอย่างของการแยกสองชั้นตามแผนที่ 18

same frame every time

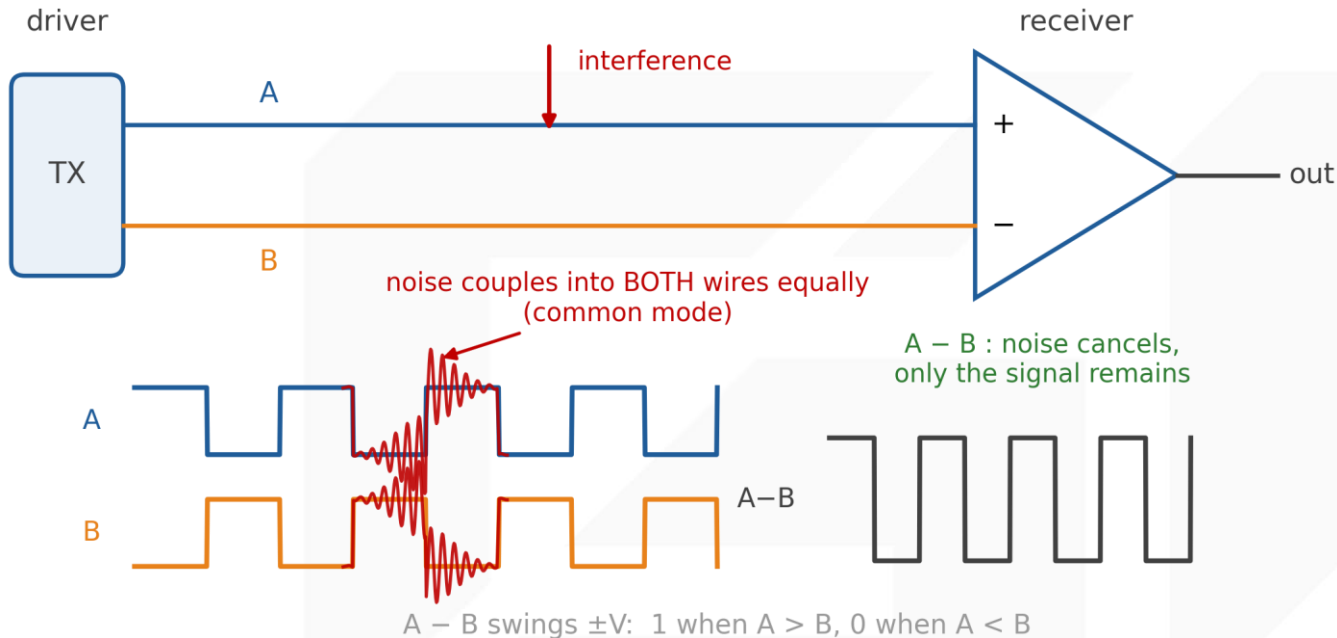


	TTL	RS-232	RS-485
logic 1	3.3 V (หรือ 5 V)	-3 ถึง -15 V	A - B < -200 mV
logic 0	0 V	+3 ถึง +15 V	A - B > +200 mV
สาย	1 เส้น/ทิศ + GND	1 เส้น/ทิศ + GND	2 เส้น A/B ใช้รวม + GND
ระยะจริง	~1 m	~15 m	~1,200 m
หลายตัวบนสายเดียว	ไม่ได้	ไม่ได้	ได้ (Modbus)
ต่อ ESP32 ตรง	ได้ (3.3 V)	ไม่ได้ — ขาพังทันที	ไม่ได้ — ผ่าน MAX485
พบในวิชา	GPS M8N · ultrasonic	เครื่องชั่งของภาควิชา	XY-MD02 · S-Soil MT-02 · S700-B

- IC แปลงระดับสัญญาณราคาถูกอย่าง MAX3232 (TTL ↔ RS-232) และ MAX485 (TTL ↔ RS-485) ทำหน้าที่เฉพาะ physical layer frame และ baud rate ไม่เปลี่ยน
- ESP32 รับแรงดันได้ไม่เกิน 3.6 V และรับแรงดันลบไม่ได้เลย ห้ามเสียบสาย RS-232 ของเครื่องชั่งเข้า GPIO โดยตรง (กิจกรรม 4.0 จะสาธิตด้วย oscilloscope ให้ดูก่อน)
- RS-485 ส่งได้ทีละทิศทาง (half-duplex) บนสายคู่เดียว จึงต้องมีขา DE/RE คอยสลับระหว่างส่งกับรับ ซึ่ง library Modbus จัดการให้อยู่แล้ว

RS-485 — Differential Signalling

ทำไมสายสองเส้นที่ส่งสัญญาณกลับด้านกันจึงส่งได้ไกลกว่าเส้นเดียวเป็นสิบเท่า เพราะ receiver อ่านผลต่างระหว่างสองเส้น ไม่ได้เทียบกับ GND



สาย 2 เส้น A กับ B (twisted pair) บวก GND รวม อุปกรณ์ทุกตัวใช้สายชุดเดียวกัน

Address RS-485 เองไม่มี protocol ที่วิ่งอยู่บนมันอย่าง Modbus เป็นคนใส่ให้

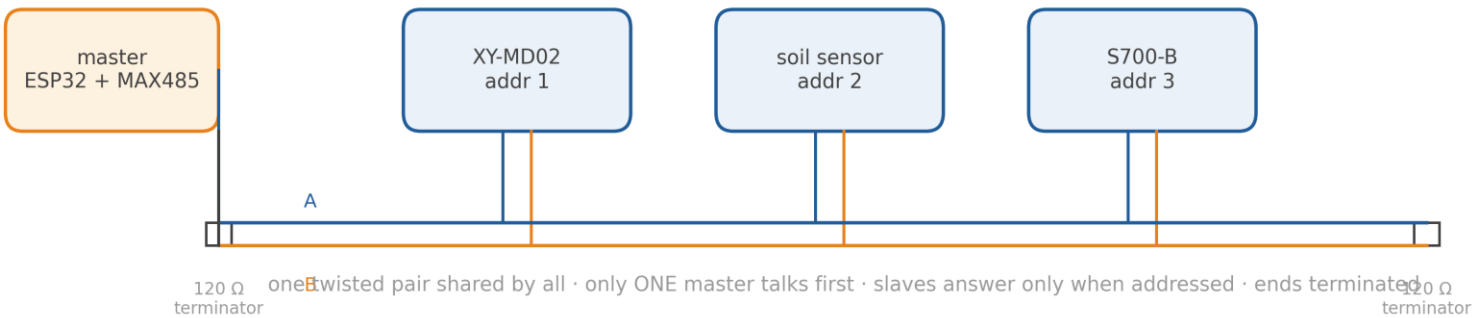
Clock เหมือน UART คือตกลง baud rate กันก่อน และเป็น half-duplex ต้องสลับทิศด้วย DE/RE

ระยะ 1,200 m ที่ความเร็วไม่เกิน 100 kbit/s ต่อได้ 32 ตัวต่อสายหนึ่งชุด (มากกว่านั้นถ้าใช้ driver รุ่นใหม่)

- terminator 120 Ω ใส่เฉพาะที่ปลายสายทั้งสองข้าง ไม่ใช่ทุกตัว เพื่อกันสัญญาณสะท้อน module MAX485 บางรุ่นมีติดมาแล้ว ต้องดูก่อน
- ยังต้องเดิน GND รวมเป็นเส้นที่สาม เพราะสัญญาณ differential หนีความต่างศักย์ระหว่าง ground ได้แค่ประมาณ ± 7 V
- S-Soil MT-02 (กลุ่ม 3) และ S700-B (กลุ่ม 5) ใช้ RS-485 ก็เพราะเดินสายจากแปลงมาที่ node ได้เป็นสิบเมตรโดยไม่ต้องกลัว noise จากบั้ง

Modbus RTU — Asking a Slave for Two Registers

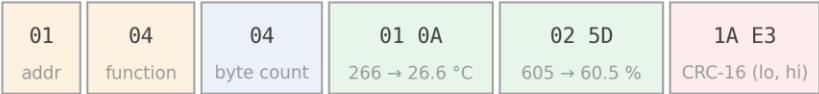
Modbus เป็น protocol บน RS-485 ที่ sensor อุตสาหกรรมและเกษตรแทบทุกรุ่นใช้ master ตัวเดียวเป็นคนถาม slave ตอบเมื่อถูกเรียกเท่านั้น



request (master → slave 1)



response (slave 1 → master)



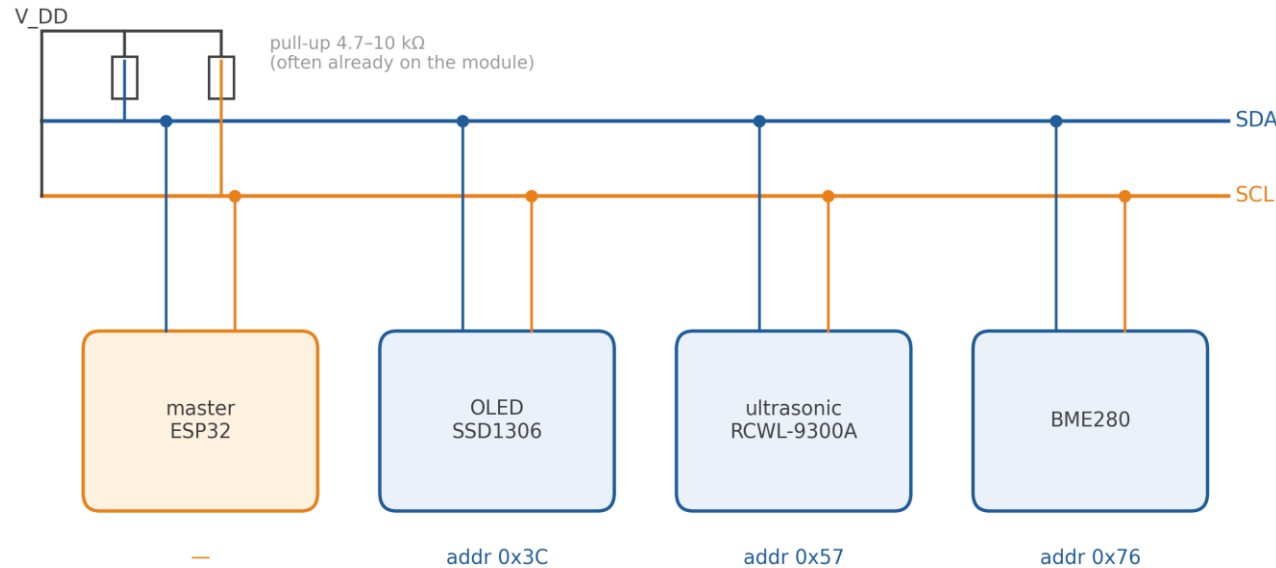
CRC-16/MODBUS (poly 0xA001, init 0xFFFF) low byte first · silence $\geq$ 3.5 char times ends a frame
register map and +10 scaling from the XY-MD02 datasheet · ModbusMaster: readInputRegisters(1, 2)

Function	ความหมาย	ใช้กับ
0x03	Read Holding Registers (register ที่อ่านและเขียนได้)	ค่าตั้งของอุปกรณ์ เช่น address และ baud rate
0x04	Read Input Registers (register ที่อ่านได้อย่างเดียว)	ค่าที่วัดได้ เช่น XY-MD02 เก็บอุณหภูมิและความชื้นไว้ที่ 0x0001–0x0002
0x06	Write Single Register	ใช้เปลี่ยน address ของ slave

- slave address (1–247) ผู้ใช้เป็นคนตั้งเอง ถ้าตั้งซ้ำกันสองตัวบนสายเดียวจะตอบพร้อมกันแล้วชนกัน ก่อนต่อ sensor ตัวใหม่เข้าสายจึงต้องตั้ง address ที่ละตัวก่อน

I²C — Two Wires, Many Devices

I²C เป็น bus สองเส้นสำหรับอุปกรณ์บนบอร์ดเดียวกัน ทุกตัวต่ออยู่บน SDA กับ SCL คู่เดียว และแยกกันด้วย address 7 บิต



every device only pulls a line LOW (open-drain); the resistor pulls it HIGH — so several devices can share the same two wires

สาย 2 เส้น คือ SDA (data) กับ SCL (clock) บวก GND ใช้ร่วมกันทุกตัว

Address มี ขนาด 7 บิต (128 ค่า) ผู้ผลิตกำหนดมา บางรุ่นเลือกได้ 2–4 ค่าด้วยขา ADDR

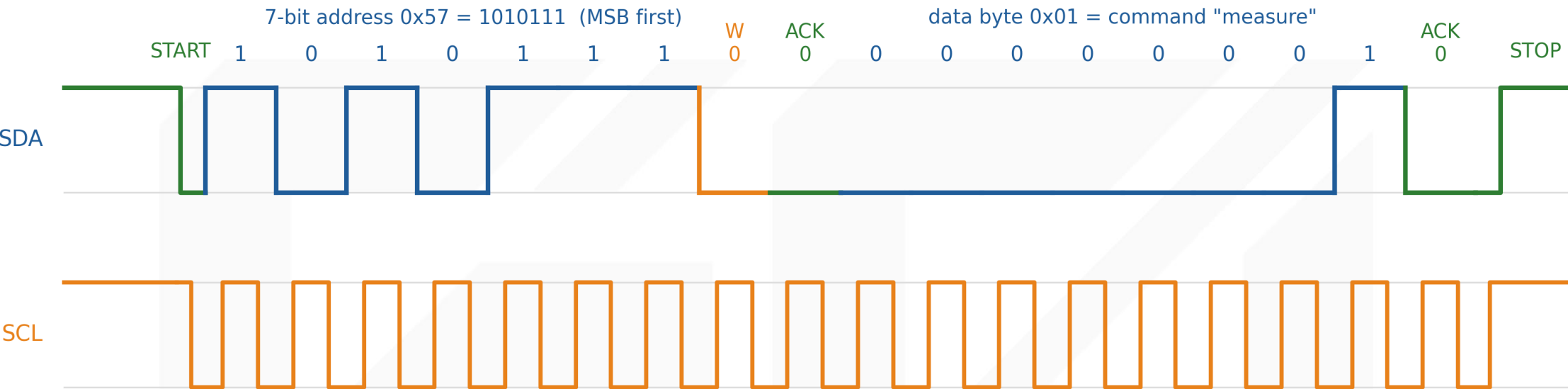
Clock master เป็นคนให้ clock บน SCL ความเร็ว 100 kHz (standard) หรือ 400 kHz (fast)

ระยะ ไม่เกิน 1 m ใช้บนบอร์ดหรือสายสั้น ๆ เพราะ capacitance ของสายทำให้ขอบสัญญาณช้าลง

- สายเป็นแบบ open-drain คืออุปกรณ์ทำได้แค่ดึงสายลง ส่วน pull-up เป็นตัวดันขึ้น จึงต่อขนานกันหลายตัวได้โดยไม่ลัดวงจร และเป็นกลไกของ ACK ด้วย (slave ดึง SDA ลงเพื่อตอบรับ)
- pull-up 4.7–10 kΩ มักมีอยู่บน module OLED และ ultrasonic แล้ว ถ้าใส่ซ้ำหลายตัวขนานกันค่ารวมจะต่ำเกินไป ให้รัน I²C scanner ก่อน ถ้าเจอทั้ง 0x3C และ 0x57 ก็ไม่ต้องเพิ่ม
- ในกิจกรรม 4.4 จะต่อ ultrasonic โหมด I²C (0x57) กับ OLED (0x3C) บนสายคู่เดียวกัน ให้จอแสดงผลอ่านค่า sensor ผ่าน bus เดียวกัน

I²C — One Transaction, Line by Line

นี่คือสิ่งที่เห็นบน logic analyzer ในกิจกรรม 4.4 และดูว่าโค้ด Wire แต่ละบรรทัดตรงกับเหตุการณ์ไหนบนสาย



START = SDA ↓ while SCL high · STOP = SDA ↑ while SCL high · otherwise SDA changes only while SCL is low · master generates SCL · 9th bit = ACK

slave 0x57 pulls SDA low during the 9th clock = "I am here" — this is exactly what an I²C scanner looks for

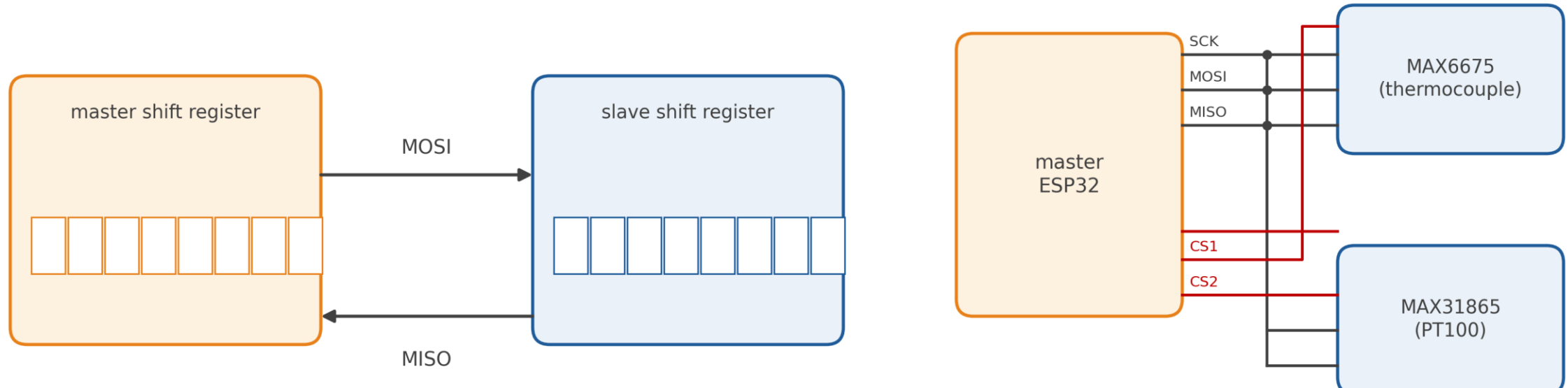
Arduino: `Wire.beginTransmission(0x57) → START + address + W · Wire.write(1) → data byte · Wire.endTransmission() → STOP`

- ตอนอ่านค่า `Wire.requestFrom(0x57, 3)` คือ START ตามด้วย address และบิต R จากนั้น master ให้ clock 3 byte slave ส่งข้อมูล master ตอบ ACK ทุก byte ยกเว้น byte สุดท้ายที่ตอบ NACK แล้วจึง STOP
- I²C scanner ทำแค่วน address 1–127 ส่ง START + address + W แล้วดูว่ามีใครตอบ ACK แค่นั้นเอง

SPI — Fastest, One CS per Device

SPI ใช้สายสี่เส้น ไม่มี address ไม่มี ACK ใต้ความง่ายและความเร็วระดับ MHz แต่ต้องใช้ขา CS หนึ่งเส้นต่ออุปกรณ์หนึ่งตัว

one exchange = 8 clocks

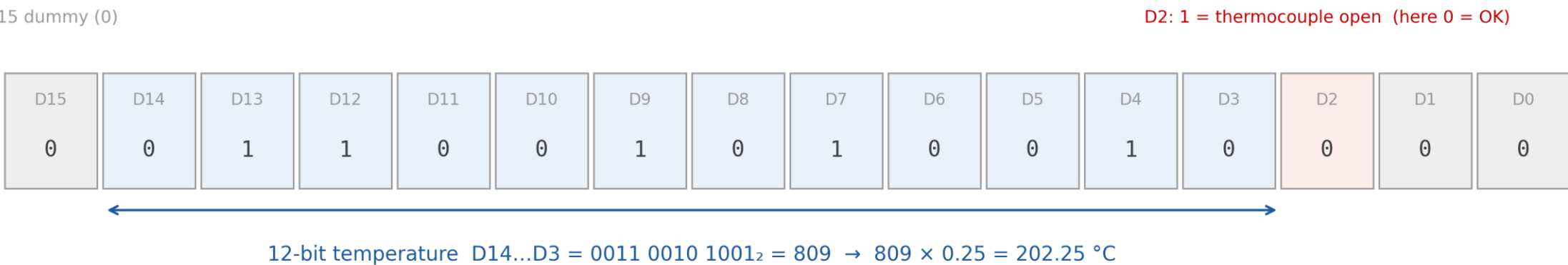


- Mode 0–3 (CPOL/CPHA) กำหนดว่าจะอ่านบิตที่ขอบขาขึ้นหรือขาลงของ clock ต้องดูจาก datasheet ของอุปกรณ์ ถ้าตั้งผิดจะได้ค่าเพี้ยนโดยไม่มี error แจ้ง
- ในวิชานี้จะเจอ SPI ที่ MAX6675 (thermocouple ชนิด K ใน Lab 6 สถานี B) และ MAX31865 (PT100 สถานี A) นอกจากนี้ microSD และจอ TFT ส่วนใหญ่ก็ใช้ SPI

สาย 4 เส้น SCK, MOSI, MISO ใช้ร่วมกัน ส่วน CS แยกตามอุปกรณ์ Address ไม่มี เลือกอุปกรณ์ด้วย CS Clock master เป็นคนให้ full-duplex 1–20 MHz หรือสูงกว่า ระยะ ไม่เกิน 30 cm บนบอร์ด

SPI — Reading a 16-bit Word from MAX6675

MAX6675 เป็นอุปกรณ์ SPI ที่ง่ายที่สุดในวิชานี้ ไม่มี register ไม่มีคำสั่ง แค่ดึง CS ลงแล้วให้ clock 16 ครั้ง ก็ได้ข้อมูล 16 บิตกลับมา



CS low → 16 SCK pulses → read 16 bits on MISO (MOSI unused) → CS high · conversion takes ~220 ms

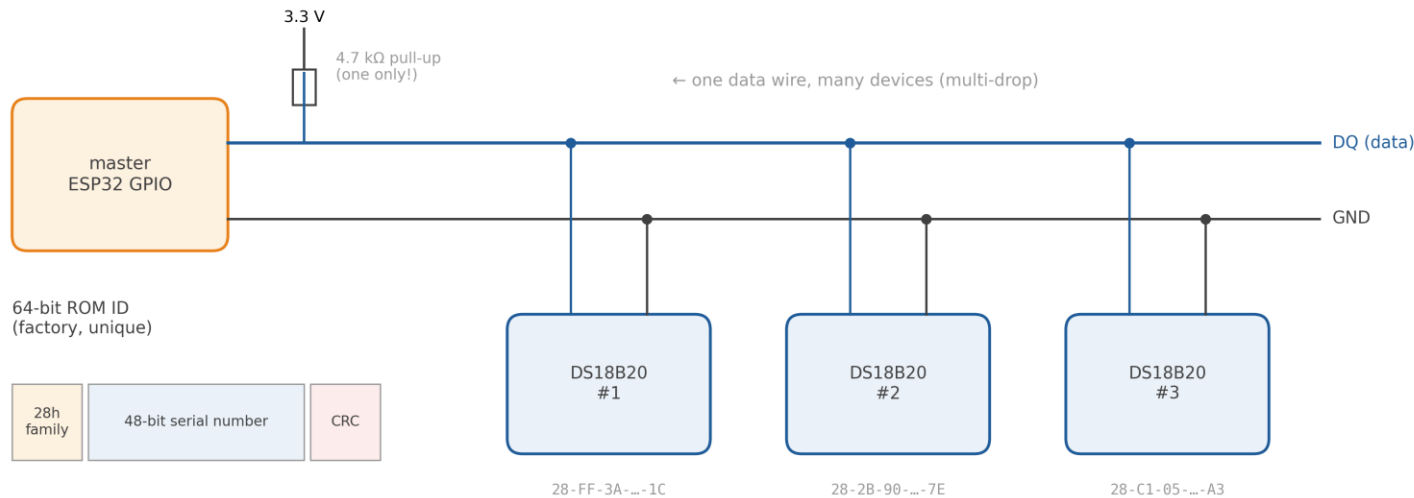
D1 = device ID · D0 = state

ขั้น	โค้ด (Arduino SPI)	บนสาย
1	digitalWrite(CS, LOW);	เลือกชิป ชิปจะเริ่มส่งบิต D15 ออกจาก MISO
2	uint16_t w = SPI.transfer16(0);	ให้ clock 16 pulse MOSI ส่งค่า 0 (ชิปไม่สนใจ) MISO รับ 16 บิต
3	digitalWrite(CS, HIGH);	ปล่อยชิป ชิปเริ่มแปลงค่าใหม่ ใช้เวลาประมาณ 220 ms
4	open = w & 0x0004; T = ((w >> 3) & 0x0FFF) * 0.25;	mask บิต 2 แล้ว shift ขวา 3 บิต คูณ 0.25 ได้ °C

- MAX31865 (PT100) ซับซ้อนกว่านิดหน่อย ต้องเขียน register ตั้งค่าก่อน แล้วอ่าน register RTD 2 byte แต่รูปแบบก็เหมือนกัน คือ CS ลง ส่งตำแหน่ง register รับข้อมูล แล้ว CS ขึ้น

1-Wire — One Wire, 64-bit ID

1-Wire เป็น bus ที่ประหยัดสายที่สุด data และไฟเลี้ยงอยู่บนเส้นเดียว ทุกตัวมี ID จากโรงงาน จึงเสียบเพิ่มได้เลยโดยไม่ต้องตั้งค่า



สาย 1 เส้น DQ บวก GND (ดึงไฟเลี้ยงจาก DQ ได้ เรียกว่า parasite power) หรือใช้ 3 เส้นเมื่อสายยาว

Address มี เป็น ROM 64 บิตที่ไม่ซ้ำกันจากโรงงาน master ค้นหาเองได้ด้วยคำสั่ง Search ROM

Clock master เป็นคนกำหนดจังหวะทั้งหมดในระดับ μs เช่น reset 480 μs หรือ slot 60 μs

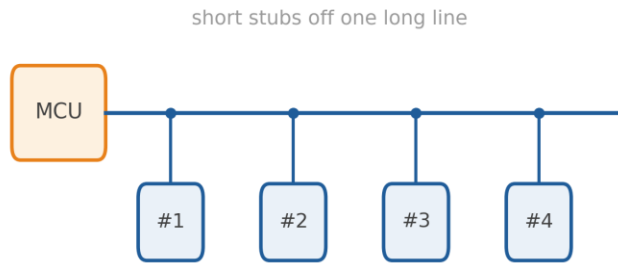
ระยะ ได้หลายสิบเมตรถ้าสายดีและ pull-up เหมาะสม แต่ช้าประมาณ 16 kbit/s

- MCU ต้องวางระหว่างรับส่ง เพราะต้องจับเวลาเองในระดับ μs เป็นปัญหาเดียวกับการจับเวลา pulse ในแผนที่ 17 library OneWire จึงปิด interrupt ช่วงขณะตอนรับส่ง
- ROM ประกอบด้วย family code 8 บิต (0x28 คือ DS18B20) หมายเลขเครื่อง 48 บิต และ CRC 8 บิต family code ใช้แยกชนิดอุปกรณ์ที่อยู่บนสายเดียวกันได้
- สังเกตว่า address แต่ละแบบมีที่มาต่างกัน Modbus ผู้ใช้ตัวเอง (เข้าได้ก็ชนได้) I²C ผู้ผลิตกำหนด (เลือกได้ไม่กี่ค่า) ส่วน 1-Wire โรงงานประทับมาให้ ไม่มีทางซ้ำ
- ใน term project ทุก node ดินจะใช้ DS18B20 2-3 ตัวที่ความลึกต่างกันบนสายเส้นเดียว เพื่อเอาอุณหภูมิไปชดเชยค่าความชื้นดิน

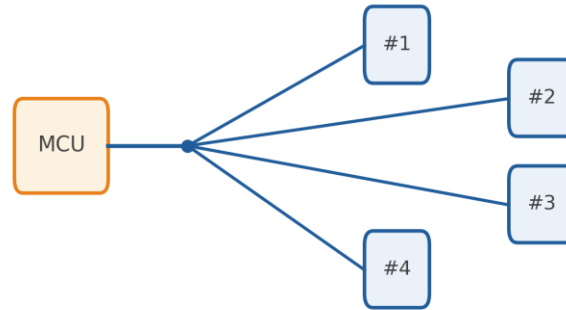
DS18B20 in the Field

ข้อควรระวังจากการใช้งานจริง และตัวอย่างว่าข้อเหมือนกันไม่ได้แปลว่าเป็นของอย่างเดียวกัน

daisy chain — recommended



star — avoid (reflections on long runs)



Resolution	ชั้นละ	เวลาแปลง
9 บิต	0.5 °C	94 ms
10 บิต	0.25 °C	188 ms
11 บิต	0.125 °C	375 ms
12 บิต	0.0625 °C	750 ms

accuracy อยู่ที่ ± 0.5 °C (ช่วง -10 ถึง 85 °C) ไม่ว่าจะตั้งกี่บิต ย้ำอีกครั้งว่า resolution กับ accuracy เป็นคนละเรื่อง (แผ่นที่ 5)

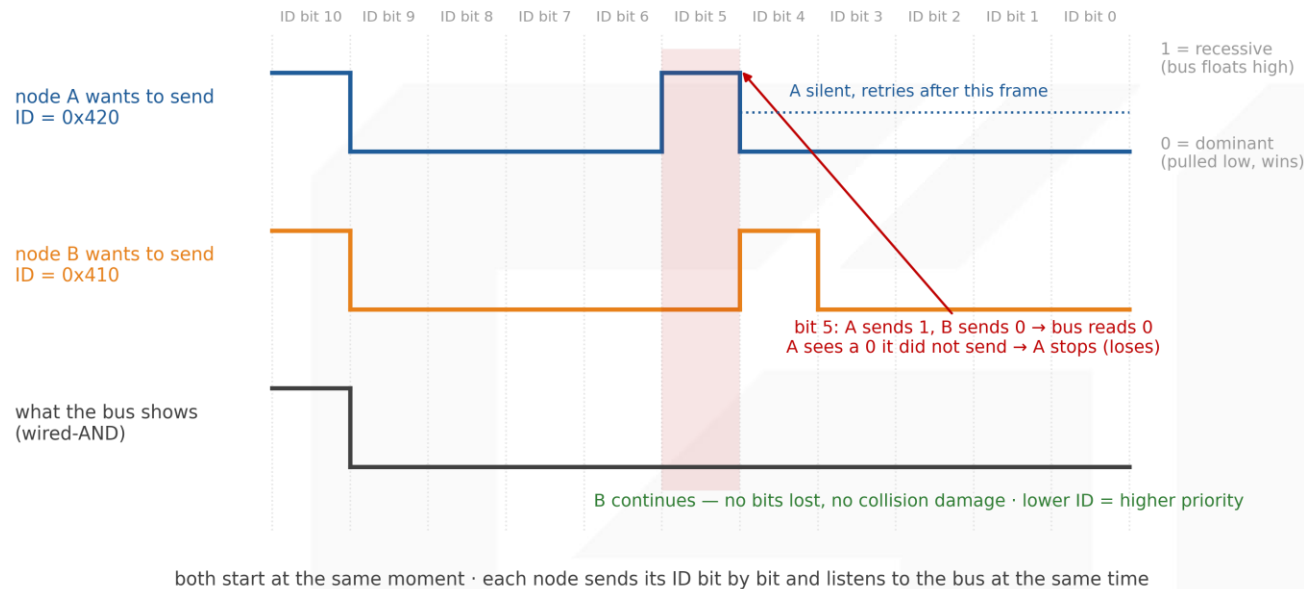
- ต่อแบบ daisy chain คือแยกสายสั้น ๆ ออกจากสายหลักเส้นเดียว อย่าต่อแบบ star ถ้าสายยาวให้ใช้ twisted pair และลด pull-up เหลือประมาณ 2.2 k Ω
- บอร์ด adapter ของภาควิชา มี pull-up 4.7 k Ω อยู่แล้ว ห้ามใส่ซ้ำ
- DS18B20 ของปลอมมีเยอะ สังเกตได้จาก ROM ที่ผิดรูปแบบหรือ accuracy หลุดสเปค ให้ตรวจเทียบกับ PT100 ที่ Lab 6 สถานี A ก่อนเอาลงแปลง

ระวังซื้อซ้ำ: บอร์ด HC-SR04 รุ่นปี 2020 มีโหมดชื่อ "1-WIRE"

โหมดนั้นแค่รวมขา Trig กับ Echo ไว้ขาดียว ไม่มี address ไม่มี ROM และไม่ใช้ protocol 1-Wire ของ Dallas/Maxim แต่อย่างใด ต้องอ่าน datasheet ไม่ใช่เดาจากชื่อ เช่นเดียวกับคำว่า "SPI-compatible" หรือ "I²C-like" ที่เจอบ่อยใน module ราคาถูก

CAN Bus — Principle

physical layer ของ CAN แทบจะเหมือน RS-485 แต่ protocol ตรงกันข้าม คือไม่มี master ทุก node ส่งได้ และเมื่อชนกันจะตัดสินด้วย priority



สาย 2 เส้น CAN_H กับ CAN_L แบบ differential มี terminator 120 Ω ที่ปลายทั้งสองข้าง (เหมือน RS-485)

Address ไม่มี address ของผู้รับ มีแต่ message ID ที่บอกกว่าข้อมูลคืออะไร ทุก node ได้ยินหมดแล้วกรองเอาเอง

Clock ไม่มี master ทุก node ส่งได้เมื่อ bus ว่าง โดยตกลง bit rate ร่วมกัน

ระยะ 1 Mbit/s ที่ 40 m, 250 kbit/s ที่ 250 m, 125 kbit/s ที่ 500 m

- การตัดสิน (arbitration) ใช้หลักว่าบิต 0 (dominant) ชนะบิต 1 (recessive) ดังนั้น ID ที่ค่าน้อยกว่าชนะ ฝ่ายแพ้จะหยุดส่งเองแล้วค่อยส่งใหม่ ไม่มีข้อมูลเสีย ส่วน CRC, ACK, error counter และการตัด node ที่เสียออก hardware ทำให้หมด

CAN frame (classic, up to 1 Mbit/s)



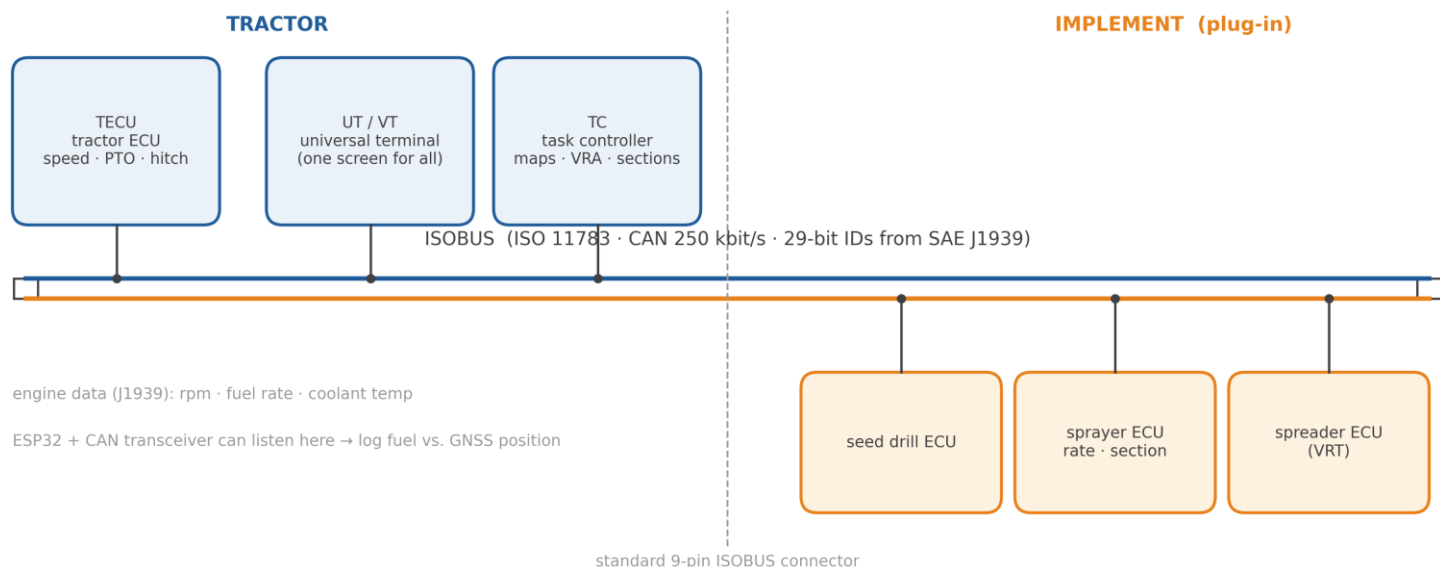
no destination address — every node hears every frame and filters by ID · CRC + ACK + error counters are done by the controller hardware, not by your code

ESP32 มี CAN controller ในตัว (เรียกว่า TWAI) เพิ่ม transceiver แคตัวเดียว (SN65HVD230 แบบ 3.3 V) ก็ต่อเข้า bus ของรถได้

เทียบกับ Modbus แล้ว Modbus กันชนด้วยการให้ master ตัวเดียวถามทีละตัว ส่วน CAN กันชนด้วย priority ใครมีเรื่องด่วนกว่าได้พูดก่อน

CAN in Agricultural Machinery

CAN คือภาษาที่รถแทรกเตอร์กับอุปกรณ์ต่อพ่วงใช้คุยกัน วิศวกรเกษตรที่ทำงานกับเครื่องจักรจะได้เจอ CAN ก่อน RS-485 เสียอีก



SAE J1939 ภาษาของเครื่องยนต์และตัวรถ เช่น รอบเครื่อง อัตราใช้น้ำมัน อุณหภูมิ น้ำ ความเร็ว ใช้ ID 29 บิตจัดกลุ่มเป็น PGN (กลุ่ม) และ SPN (ค่าย่อย) ที่ 250 kbit/s

ISOBUS (ISO 11783) ต่อยอดจาก J1939 สำหรับให้ tractor คุยกับ implement UT/VT คือจอเดียวที่คุมได้ทุกอุปกรณ์ TC สั่ง VRA และ section control ตามแผนที่ ส่วน TECU ส่งความเร็ว PTO และระยะยกจากตัวรถ

ป้ายรับรอง AEF UT, TC-BAS, TC-GEO, TC-SC เป็นตัวบอกว่าอุปกรณ์ต่างยี่ห้อคุยกันได้จริงในเรื่องไหนบ้าง

ในวิชา ไม่มีใน lab แต่ใช้ทักษะชุดเดียวกัน เช่น เอา ESP32 กับ transceiver ไปดักอ่าน J1939 แล้วบันทึกอัตราใช้น้ำมันคู่กับพิกัด GNSS เป็นโครงการที่ทำได้ด้วยความรูจาก INS03

- สรุป Modbus กับ CAN ในประโยคเดียว Modbus คือ master ถามทีละตัว (polling) เหมาะกับ sensor ที่ค่าเปลี่ยนช้า ส่วน CAN เป็น multi-master แบบ event-driven เหมาะกับเครื่องจักรที่มี controller หลายตัวต้องพูดพร้อมกัน
- ทั้งสองแบบใช้สายคู่ differential กับ terminator 120 Ω เหมือนกัน ยืนยันประโยค ① ในแผ่นที่ 18 ว่าระยะทางเป็นเรื่องของ physical layer

Choosing an Interface — One Table

ตารางเดียวกับที่อยู่หัวกิจกรรมในใบงาน Lab 4 และ 6 ใช้คำถามสี่ข้อเดิม คือ กี่เส้น มี address ไหม ใครให้ clock และส่งได้ไกลแค่ไหน

Interface	สาย	Address	ใครให้ clock	ระยะจริง	ใช้ที่ไหนในวิชานี้
GPIO / จับเวลา	1–2	ไม่มี	ผู้ใช้เขียนเอง	< 1 m	HC-SR04 trig/echo · flow pulse · relay
ADC (analog)	1	ไม่มี	ผู้ใช้สั่งอ่าน	สั้นที่สุด (noise)	Lab 1–3 · NTC · LDR · MPX5010DP · capacitive
UART — TTL	2 (TX/RX)	ไม่มี (1:1)	ตกลง baud · ไม่มี clock	~1 m	GPS M8N · ultrasonic โหมด UART
UART — RS-232	2 + GND	ไม่มี	เหมือน TTL	~15 m	เครื่องชั่งของภาควิชา
UART — RS-485 / Modbus	2 (differential)	มี (slave ผู้ใช้ตั้ง)	master ถาม slave ตอบ	~1,200 m	XY-MD02 · S-Soil MT-02 · S700-B
I ² C	2 (SDA/SCL)	มี (7 บิต ผู้ผลิตกำหนด)	master ให้ clock	< 1 m	OLED 0x3C · ultrasonic 0x57
SPI	4 (SCK/MOSI/MISO/CS)	ไม่มี — ใช้ CS แยกตัว	master ให้ clock · เร็วที่สุด	< 30 cm	MAX6675 · MAX31865 · microSD
1-Wire	1 + GND	มี (ROM 64 บิตจากโรงงาน)	master กำหนดระดับ μ s	หลายสิบเมตร	DS18B20 — Lab 6 A · ทุก node ดิน
CAN (ไม่มีใน lab)	2 (differential)	ไม่มีผู้รับ — message ID	multi-master · priority	40 m @ 1 Mbit/s	รถแทรกเตอร์ · ISOBUS

- ① **address แยก bus จาก point-to-point** RS-485, I²C และ 1-Wire ต่อหลายตัวบนสายชุดเดียวได้ ส่วน TTL UART ทำไม่ได้ SPI ไม่มี address แต่ใช้ CS แทน ได้ความเร็วแต่เปลืองขา
- ② **ที่มาของ address ต่างกัน** Modbus ผู้ใช้ตั้งเอง (เข้าได้ก็ชนได้) I²C ผู้ผลิตกำหนด 1-Wire โรงงานประทับมาให้ ③ **ระยะ = physical layer** SPI กับ I²C เร็วแต่ส่งได้ใกล้ RS-485 กับ 1-Wire ออกแบบมาให้ลากสายยาว ซึ่งเป็นสิ่งที่งานในแปลงต้องการ

Next Week — Lab 4 Preview and Pre-lab

Lab 4 (Digital Signals กับ ESP32) เลื่อนไปสัปดาห์ที่ 12 วันนี้ให้ทำ Lab 2 และ 3 ต่อให้เสร็จ

กิจกรรม	Interface	แผ่นที่เกี่ยวข้อง	สิ่งที่จะได้เห็น
4.0 เครื่องชั่ง RS-232 (สาธิต)	UART / RS-232	19–21	สัญญาณ $\pm 12\text{ V}$ บน oscilloscope ข้อความ ASCII และสถานะ ST/US
4.1 ADC ของ ESP32	ADC · attenuation	13–16	calibrate 6 จุดเทียบกับ DMM แล้วหาค่า m และ c
4.2 GPS fix quality	UART / TTL	19–20	อ่านค่า fix quality ก่อนเอาตำแหน่งไปใช้
4.3 ultrasonic โหมด UART	UART / binary	19–20	ถอด byte ด้วยมือจาก logic analyzer
4.4 ultrasonic I ² C + OLED	I ² C	24–25	สแกนเจอ 0x3C กับ 0x57 และดู transaction บนสาย
4.5 XY-MD02	RS-485 / Modbus	21–23	สัญญาณ differential และ frame ถ้าม-ตอบ
4.8 relay + deadband	GPIO	—	จากการวัดไปสู่การสั่งงาน ครั้งแรกของวิชา

Pre-lab — ต้องเสร็จก่อนคาบสัปดาห์ที่ 12

- ติดตั้ง Arduino IDE พร้อม ESP32 board package และ driver CP2102 หรือ CH340
- ทดสอบ Blink บน ESP32 DevKit ให้ผ่าน แล้วส่งภาพหน้าจอมา
- อ่านแผ่นที่มีป้าย "อ่านก่อน Lab" (20, 23, 25, 27, 29) และตารางในแผ่นที่ 32
- โค้ด skeleton ของทุกกิจกรรมจะแจกล่วงหน้า ให้เติมเฉพาะส่วนที่เว้นไว้

เครื่องมือสามชิ้น ตอบคำถามสามข้อ

- Logic analyzer ตอบว่า "บนสายมีอะไรวิ่งอยู่" ด้วยรูปคลื่นจริงของทุก interface
- USB-TTL ตอบว่า "sensor พูดว่าอะไร" อ่าน UART ได้โดยไม่ต้องเขียนโค้ด
- USB-RS485 ตอบว่า "slave ตอบไหม" ทดสอบ Modbus จากคอมพิวเตอร์ก่อนต่อกับ ESP32

Appendix

ที่มาของสมการ DAC และ R-2R, Flash, Pipeline, $\Delta\Sigma$, Dual-slope และตารางเปรียบเทียบ จากเดิม Digital_Signals v6 สำหรับอ่านเพิ่มเติม ไม่ใช่ในคาบ

A1 · DAC — current derivation

- Digital-to-Analog Converter (DAC, D/A)

$$\begin{aligned}
 V_o &= \frac{i_f R}{2} \quad ; i_f = -(i_3 + i_2 + i_1 + i_0) \\
 &= -(i_3 + i_2 + i_1 + i_0) \frac{R}{2} \\
 &= -\left(\frac{b_3}{R} + \frac{b_2}{2R} + \frac{b_1}{4R} + \frac{b_0}{8R}\right) \frac{R}{2} \cdot V_{ref} \\
 &= -\left(\frac{b_3}{2} + \frac{b_2}{4} + \frac{b_1}{8} + \frac{b_0}{16}\right) V_{ref} \\
 &= -\left(\frac{8b_3 + 4b_2 + 2b_1 + b_0}{16}\right) V_{ref} \\
 V_o &= -\left(\frac{2^3 b_3 + 2^2 b_2 + 2^1 b_1 + 2^0 b_0}{2^4}\right) V_{ref}
 \end{aligned}$$

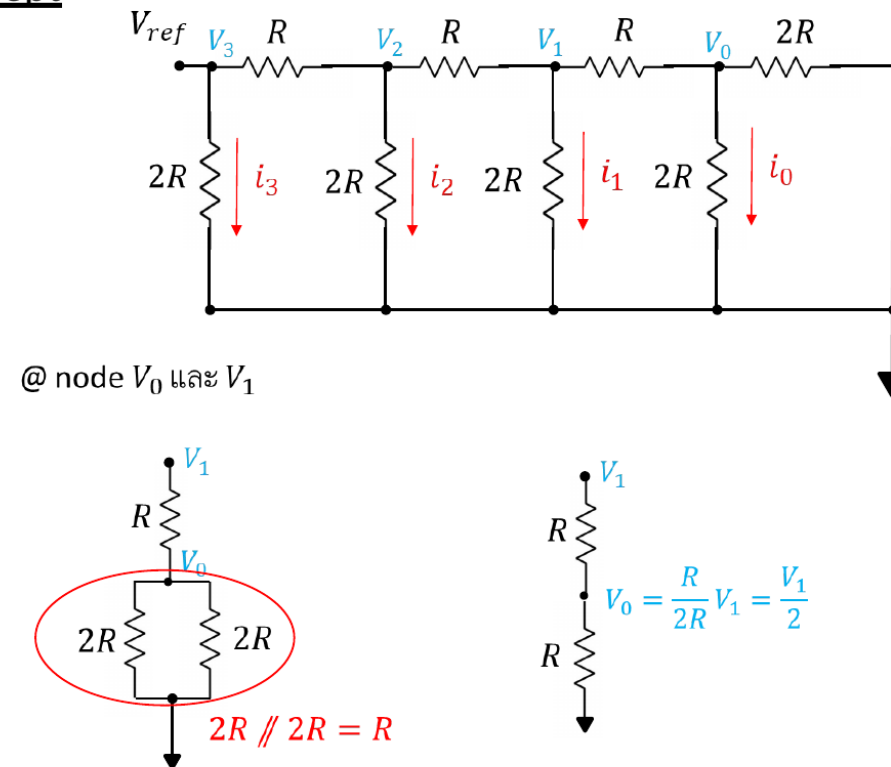
ปัญหา ต้องใช้ R
 ต่างขนาดกันมาก
 เช่นในวงจร DAC
 ขนาด 16-bit
 R ตัวใหญ่สุด = 2^{15}
 R
 R ตัวเล็กสุด =
 $R/2$
 ต่างกันอยู่ 2^{16} เท่า
 (65536)

8

A2 · R-2R ladder — concept

- R-2R Ladder Network (DAC)

Concept



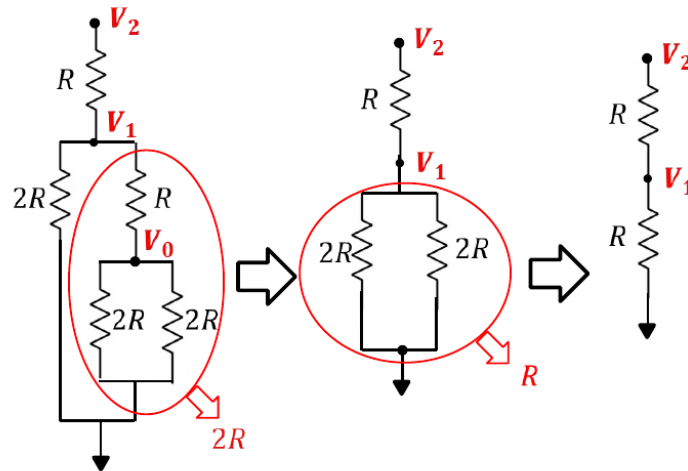
9

ภาพจากเค็ด Digital_Signals_v6 ของผู้สอน ฉบับที่มี animation อยู่ในไฟล์ต้นฉบับ

A3 · R-2R ladder — node analysis

• R-2R Ladder Network (DAC)

@ node V_1 และ V_2



$$V_1 = \frac{R}{2R} V_2 = \frac{V_2}{2}$$

$$V_2 = \frac{V_3}{2} = \frac{V_{ref}}{2}$$

พิจารณา

กระแส

$$i_3 = \frac{V_3}{2R} = \frac{V_{ref}}{2R}$$

$$i_2 = \frac{V_2}{2R} = \frac{V_{3/2}}{2R} = \frac{V_{ref}}{4R}$$

$$i_1 = \frac{V_1}{2R} = \dots = \frac{V_{ref}}{8R}$$

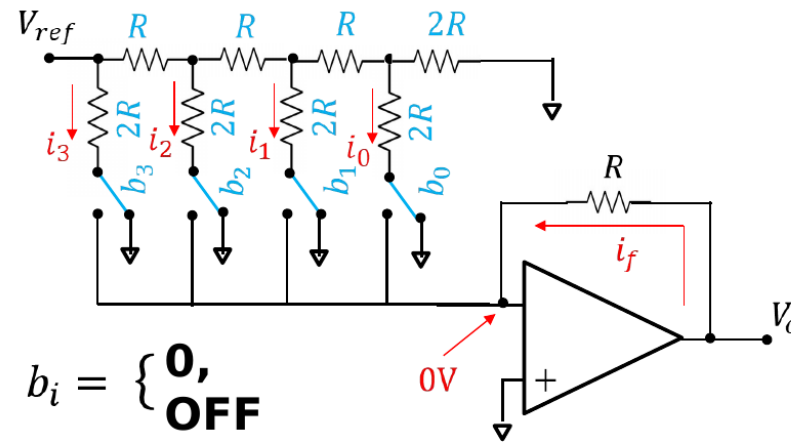
$$i_0 = \frac{V_0}{2R} = \dots = \frac{V_{ref}}{16R}$$

10

ภาพจากเดิม Digital_Signals_v6 ของผู้สอน ฉบับที่มี animation อยู่ในไฟล์ต้นฉบับ

A4 · R-2R ladder — result

- R-2R Ladder Network (DAC)



$$b_i = \begin{cases} 0, & \text{OFF} \\ 1, & \text{ON} \end{cases}$$

$$\begin{aligned} V_o &= -(i_3b_3 + i_2b_2 + i_1b_1 + i_0b_0)R \\ &= -\left(\frac{b_3}{2R} + \frac{b_2}{4R} + \frac{b_1}{8R} + \frac{b_0}{16R}\right)R \cdot V_{ref} \\ &= -\frac{8b_3 + 4b_2 + 2b_1 + b_0}{16} V_{ref} \\ &= -\frac{2^3b_3 + 2^2b_2 + 2^1b_1 + 2^0b_0}{2^4} V_{ref} \end{aligned}$$

⇒ ใช้ R เพียง 2 ขนาด คือ R และ 2R ไม่จำเป็นการแปลงก็ bit ก็ตาม

11

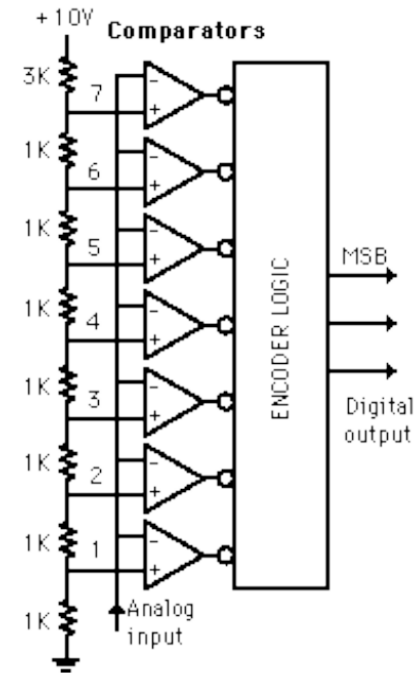
ภาพจากเค็ด Digital_Signals_v6 ของผู้สอน ฉบับที่มี animation อยู่ในไฟล์ต้นฉบับ

A5 · Flash ADC

Flash ADC

- ใช้ comparator เท่ากับจำนวน step - 1
- กรณี 16-bit ($2^{16} = 65536$ steps)
 - ใช้ comparators 65535 ตัว
 - ใช้วงจรแปลง logic ขนาดใหญ่แปลงสัญญาณจาก comparators ให้เป็น binary ขนาด 16 bit
- ใช้เวลาแปลงสัญญาณเพียง 1 cycle
- Fast, but Expensive!

$$V_{@step i} = V_i = (\sum_{i=1}^i R/R_{total}) \times V_{ref}$$



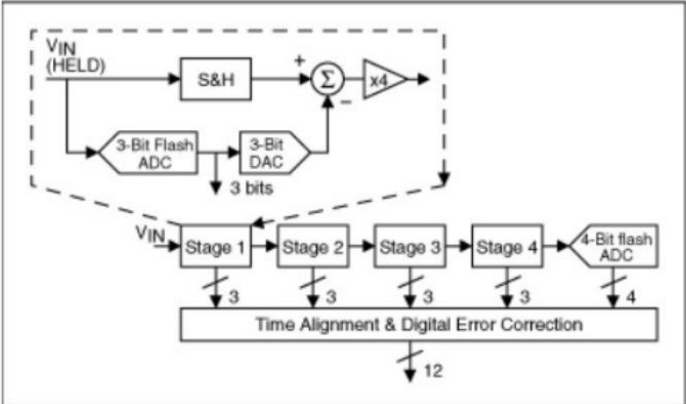
<http://hyperphysics.phy-astr.gsu.edu/hbase/Electronic/adc.html>

13

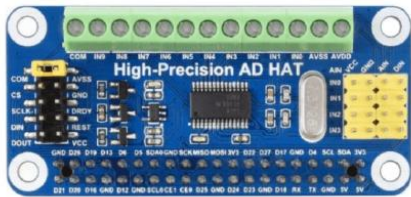
ภาพจากเด็ก Digital_Signals_v6 ของผู้สอน ฉบับที่มี animation อยู่ในไฟล์ต้นฉบับ

A6 · Pipeline ADC

- Pipeline ADC
 - ADC that divides the conversion task into several consecutive stages allowing for fast and efficient conversion



Example of 3-bit x 4-stage = 12-bit pipelined ADC



10-Channel 32-bit ADC HAT for Raspberry Pi
 ★★★★★ 3 Ratings
 Brands WaveShare Code: HAT-ADC-1032 MPN: 18983
 THB1,780.00 (VAT included)

<https://www.analog.com/en/resources/technical-articles/understanding-pipelined-adcs.html>

Output 12-Bit:	11	10	9	8	7	6	5	4	3	2	1	0
Stage 1 (3-bit):	b2	b1	b0									
Stage 2 (3-bit):			b2	b1	b0							
Stage 3 (3-bit):					b2	b1	b0					
Stage 4 (3-bit):							b2	b1	b0			
Final Flash (4-bit):									q3	q2	q1	q0
Output (Sum for Total):	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0

A7 · Delta-sigma ADC

- Delta-sigma ADC ($\Delta\Sigma$ ADC)
 - ADC that uses $\Delta\Sigma$ modulator to convert an analog input into a digital bit stream, while the digital filter translates the bit stream into a data word that represents the magnitude of the analog input.

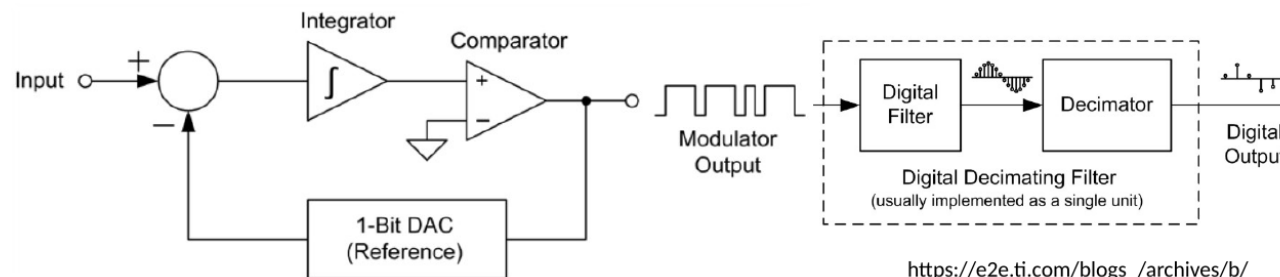


Figure 1: Delta-sigma modulator internal block diagram

https://e2e.ti.com/blogs_/archives/b/precisionhub/posts/my-new-years-resolution-benefit-from-the-high-resolution-delta-sigma-adcs

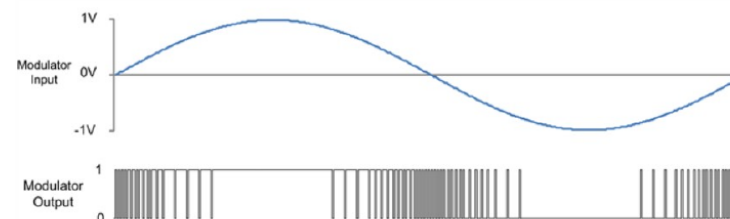


Figure 2: Sine wave input (top) and modulated output bit stream (bottom)

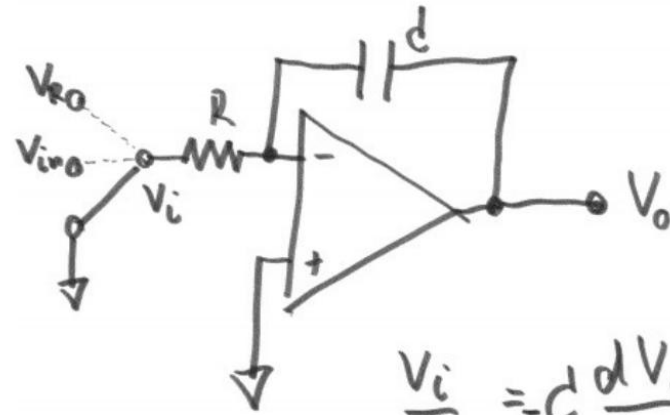
https://e2e.ti.com/blogs_/archives/b/precisionhub/posts/delta-sigma-adc-basics-understanding-the-delta-sigma-modulator

18

ภาพจากเด็ก Digital_Signals_v6 ของผู้สอน ฉบับที่มี animation อยู่ในไฟล์ต้นฉบับ

A8 · Dual-slope ADC

Dual-Slope Integration ADC



$$\frac{V_i}{R} = -C \frac{dV_o}{dt}$$

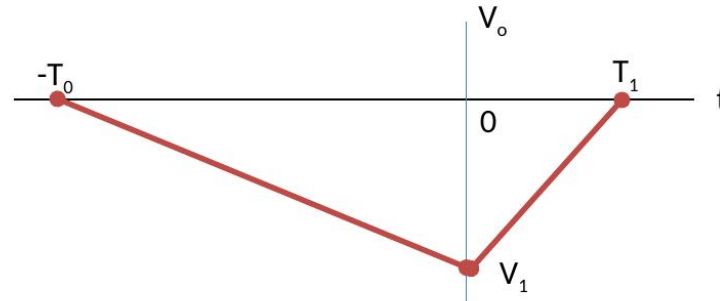
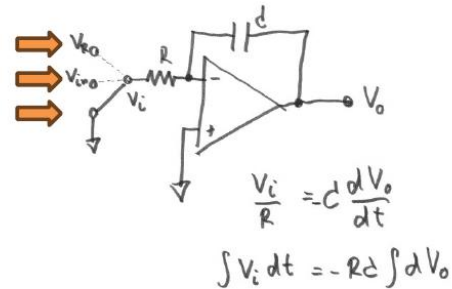
$$\int V_i dt = -RC \int dV_o$$

- ใช้วงจร Integrator เก็บประจุใน Capacitor โดย $V_i = V_{in}$
- แล้ววัดระยะเวลาที่ใช้ในการคายประจุ
- เวลาที่คายประจุแปรผันกับ V_{in} ที่ป้อน

19

ภาพจากเค็ด Digital_Signals_v6 ของผู้สอน ฉบับที่มี animation อยู่ในไฟล์ต้นฉบับ

A9 · Dual-slope — derivation (1)



- ① switch is at GND ($V_i = 0$)
 $\Rightarrow V_o = 0$
 capacitor is completely discharged

- ② switch is set to the measured voltage ($V_i = V_{in}$) for a period of T_0
 $\int_{-T_0}^0 V_{in} dt = -RC \int_0^{V_1} dV_o$
 $V_{in}(0 - (-T_0)) = -RC(V_1 - 0)$
 $V_{in} T_0 = -RC V_1 \quad (1)$

- ③ switch is then moved to V_R ($V_i = V_R$) and wait until V_o is fully discharged ($V_o = 0$)
 $\int_0^{T_1} V_R dt = -RC \int_{V_1}^0 dV_o$
 $V_R(T_1 - 0) = -RC(0 - V_1)$
 $V_R T_1 = RC V_1 \quad (2)$

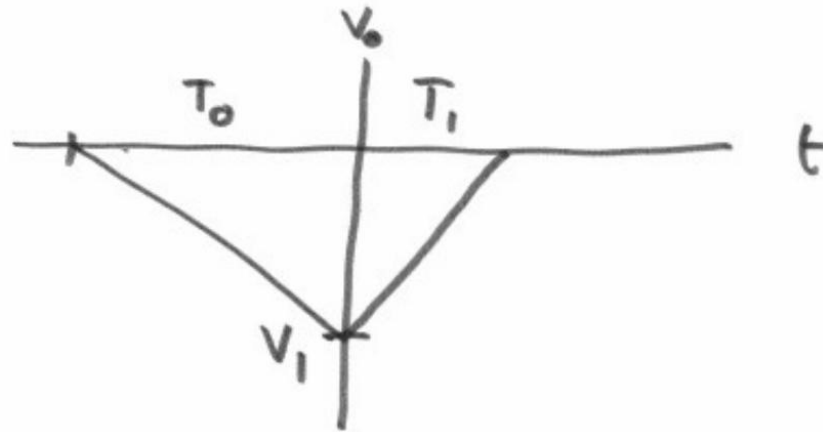
- ④ (1)/(2)

$$\frac{V_{in} T_0}{V_R T_1} = \frac{-RC V_1}{RC V_1}$$

$$V_{in} = -\frac{T_1}{T_0} V_R$$

20

A10 · Dual-slope — derivation (2)



V_{in} เป็นระดับช่วงระหว่าง T_1 กับ T_0
 หารกับ V_R
 เนื่องจาก T_0 กับ V_R คงที่
 → สามารถใช้ timer เพื่อวัดเวลา
 T_1 แล้วมาคำนวณหา V_{in} ได้

⇒ ปรอทถูก ผิดใช้กับ
 low-cost digital multimeter
 Accuracy จะอยู่กับ ความแม่นยำ
 ของ timer.

21

ภาพจากเคส Digital_Signals_v6 ของผู้สอน ฉบับที่มี animation อยู่ในไฟล์ต้นฉบับ

A11 · Comparison of ADC architectures

Comparison of ADC

Characteristic	Flash	Pipelined	SAR	Sigma-Delta
Throughput	1	2	3	4
Resolution (ENOB)	4	3	2	1
Latency	1	3	2	4
Suitability for converting multiple signals per ADC	1	2	1	3
Capability to convert non-periodic multiplexed signals	1	2	1	3
Simplified anti-aliasing				*
Can undersample	*	*	*	
Can increase resolution through averaging (with dither noise)	*	*	*	

- A rank of 1 in a performance category indicates that the architecture is inherently better than the others in that category are.
- An * indicates that the architecture has the capability or characteristic listed.

Source:

<https://www.analog.com/en/resources/analog-dialogue/articles/analog-to-digital-converter-architectures-and-choices.html>

ภาพจากเดิม Digital_Signals_v6 ของผู้สอน ฉบับที่มี animation อยู่ในไฟล์ต้นฉบับ